



MINIMUM VERTEX COVER

ROADMAP:

1. IDEA DEL PROGETTO
2. ALGORITMI CONFRONTATI
3. METODOLOGIA
4. RISULTATI PIÙ SIGNIFICATIVI
5. CONCLUSIONI

ROADMAP:

1. IDEA DEL PROGETTO

2. ALGORITMI CONFRONTATI

3. METODOLOGIA

4. RISULTATI PIÙ SIGNIFICATIVI

5. CONCLUSIONI

IDEA

Il progetto si pone l'obiettivo di analizzare e confrontare le performance di vari algoritmi per il problema del **minimum vertex cover** su grafo non orientato.

Vengono presi in esame sia la variante originale del problema che il **minimum weighted vertex cover**.

CONCETTO DI COVER

Dato un grafo, una **cover** è un **sottoinsieme S dei vertici** tale che, per ogni arco del grafo, **almeno uno dei suoi vertici** appartiene ad S.

$$G = (V, E)$$

$$S \subseteq V : \forall e = \{u, v\} \in E \Rightarrow (u \in S) \vee (v \in S)$$

COVER MINIMA

- **Sottoinsieme S di cardinalità minima**
- **Sottoinsieme S di peso totale minore**
 - Peso totale = somma dei pesi di tutti i vertici presenti nella cover

ROADMAP:

1. IDEA DEL PROGETTO

2. ALGORITMI CONFRONTATI

3. METODOLOGIA

4. RISULTATI PIÙ SIGNIFICATIVI

5. CONCLUSIONI

VERTEX COVER NON PESATO

Algoritmo 1 (si parte da un vertice qualsiasi)

0. Input: $G = (V, E)$ #grafo
1. $C = \{\}$; #cover
2. $I = E$; #archi “scoperti”
3. for v in V {
4. $C.insert(v)$;
5. $I = I / \delta(v)$;
6. if ($I.isEmpty$) then return C ;
7. }

VERTEX COVER NON PESATO

Algoritmo 2 (si ordinano i vertici per grado decrescente)

0. Input: $G = (V, E)$ #grafo
1. $W = \text{sort_by_deg}(V)$; #vertici ordinati per grado decrescente
2. return Algoritmo_1(W, E)

VERTEX COVER NON PESATO

Algoritmo 3 (basato sul matching massimale)

```
0. Input:  $G = (V, E)$  #grafo
1.  $C = \{\}$ ; #cover
2.  $I = E$ ; #archi "scoperti"
3. while( ! I.isEmpty){
4.      $e = I[0]$ ; #prendo un arco scoperto
5.      $u = \text{first\_vertex}(e)$ ;
6.      $v = \text{second\_vertex}(e)$ ;
7.      $C.\text{insert}(u)$ ;
8.      $C.\text{insert}(v)$ ;
9.      $J = e \cup \delta(u) \cup \delta(v)$ ;
10.     $I = I / J$ ;
11. }
12. return C;
```

VERTEX COVER NON PESATO

Algoritmo 4 (doubling and rounding down)

Doubling and rounding down effettuato dopo aver risolto:

- **Rilassamento lineare**
- Rilassamento lineare con odd cycle inequalities

VERTEX COVER NON PESATO

Algoritmo 4 (doubling and rounding down)

$$\min \sum_{u \in V} x_u$$

$$x_u + x_v \geq 1 \quad \forall (u,v) \in E$$

$$x_u \geq 0 \quad \forall u \in V$$

0. Input: $G = (V, E)$
1. $x^* = \text{solver_relaxed_min_vertex_cover}(G);$
2. $w = 0^{|V|};$
3. for $(i \leftarrow 1 \text{ to } |V|)$ {
4. if $(x_i^* \geq \frac{1}{2})$ then $w_i = 1;$
5. }
6. return $w;$

VERTEX COVER NON PESATO

Algoritmo 4 (doubling and rounding down)

Doubling and rounding down effettuato dopo aver risolto:

- Rilassamento lineare
- **Rilassamento lineare con odd cycle inequalities**

ODD CYCLE INEQUALITIES

Concetto: in un grafo $G = (V, E)$, S è una cover solo se $V \setminus S$ è un set indipendente (non ammette coppie di vertici adiacenti). Inoltre, a partire da un ciclo di lunghezza dispari $(2k+1)$, **non è possibile scegliere più di k vertici** senza che ci sia almeno una coppia di vertici adiacenti tra quelli selezionati.

Osservazione: da un ciclo di $2k+1$ vertici, massimo k possono formare un set indipendente, i restanti **$k+1$ formeranno una cover.**

ODD CYCLE INEQUALITIES

Conseguenza: dato un ciclo C di lunghezza $2k+1$

$$\sum_{i \in C} x[i] \geq k + 1$$

Considerazione: nel modello di PLI la $x[i]$ relativa al vertice i vale 1 se quel vertice è stato inserito nella cover. Nel modello rilassato la $x[i]$ può assumere valori frazionari e **potrebbero essere violate** le odd cycle inequalities.

ODD CYCLE INEQUALITIES

Applicazione: si potrebbe aggiungere un vincolo in più al modello di PL per ogni odd cycle inequality che deriva da un ciclo di lunghezza dispari.

Implementazione: ci si è concentrati solo sui cicli dispari di lunghezza minore possibile (triangoli).

$\forall u, v, w$ vertici di un triangolo, deve valere:

$$X[u] + x[v] + x[w] \geq 2$$

VERTEX COVER NON PESATO

Algoritmo 4 (doubling and rounding down con odd cycle inequalities)

```
0. Input:  $G = (V, E)$ 
1.  $violated = true;$ 
2.  $x^* = 0;$ 
3. while( $violated$ ){
4.    $x^* = \text{solver\_relaxed\_min\_vertex\_cover}(G);$ 
5.    $I = \{\};$  #insieme di odd cycle inequalities violate
6.    $I = \text{odd\_cycle\_violated\_by}(x^*);$ 
7.   if( $I.isEmpty$ ) {
8.      $violated = false;$ 
9.   } else {
10.     $\text{add\_to\_solver\_constrains}(I);$ 
11.  }
12. }
13.  $w = 0^{|V|};$ 
14. for ( $i \leftarrow 1$  to  $|V|$ ) {
15.   if( $x_i^* \geq \frac{1}{2}$ ) then  $w_i = 1;$ 
16. }
17. return  $w;$ 
```

VERTEX COVER NON PESATO

Algoritmo 5 (basato sul duale del rilassamento lineare)

0. Input: $G = (V, E)$
1. $x = 0^{|V|}$;
2. $y = 0^{|E|}$;
3. $I = E$; #archi “scoperti”
4. while($x \neq \text{vertex_cover}(G)$){
5. $e = I[0]$; #prendo un arco scoperto
6. $u = \text{first_vertex}(e)$;
7. $v = \text{second_vertex}(e)$;
8. $\text{increment_u} = 1 - \sum_{e \in \delta(u)} y_e$;
9. $\text{increment_v} = 1 - \sum_{e \in \delta(v)} y_e$;
10. if($\text{increment_u} < \text{increment_v}$) {
11. $y_e = 0$;
12. $x_u = 1$;
13. $I = I / \delta(u)$;

VERTEX COVER NON PESATO

Algoritmo 5 (basato sul duale del rilassamento lineare)

```
14. } else if(increment_u > increment_v){
15.     y_e = 0;
16.     x_v = 1;
17.     I = I /  $\delta(v)$ ;
18. } else { #posso scegliere uno dei due
19.     y_e = increment_u; #poiché sono uguali
21.     if(deg(u)  $\leq$  deg(v)){ # scelgo il vertice con il grado maggiore
22.         x_v = 1;
23.         I = I /  $\delta(v)$ ;
24.     } else {
25.         x_u = 1;
26.         I = I /  $\delta(u)$ ;
27.     }
28. }
29. }
30. return x;
```

VERTEX COVER PESATO

Algoritmo 1 (si ordinano i vertici per peso crescente)

```
0. Input:  $G = (V, E)$  #grafo
1.  $C = \{\}$ ; #cover
2.  $I = E$ ; #archi "scoperti"
3.  $W = \text{sort\_by\_weight}(V)$ ; #vertici ordinati per peso crescente
4. for  $v$  in  $W$  {
5.      $C.\text{insert}(v)$ ;
6.      $I = I / \delta(v)$ ;
7.     if ( $I.\text{isEmpty}$ ) then return  $C$ ;
8. }
```

VERTEX COVER PESATO

Algoritmo 2 (si ordinano i vertici per rapporto peso/grado crescente)

```
0. Input:  $G = (V, E)$  #grafo
1.  $C = \{\}$ ; #cover
2.  $I = E$ ; #archi "scoperti"
3.  $W = \text{sort\_by\_deg\_and\_weight}(V)$ ;
3. for  $v$  in  $W$  {
4.    $C.\text{insert}(v)$ ;
5.    $I = I / \delta(v)$ ;
6.   if ( $I.\text{isEmpty}$ ) then return  $C$ ;
7. }
```

VERTEX COVER PESATO

Algoritmo 3 (si ordinano gli archi per costo)

```
0. Input:  $G = (V, E)$  #grafo
1.  $C = \{\}$ ; #cover
2.  $I = E$ ; #archi "scoperti"
3. while( ! I.isEmpty ){
4.    $W = \text{sort\_by\_weight}(I)$ ; #ordino gli  $e = (u,v) \in E$  in ordine crescente di  $c_u + c_v$ 
5.    $e = W[0]$ ; #prendo il minimo arco scoperto
6.    $u = \text{first\_vertex}(e)$ ;
7.    $v = \text{second\_vertex}(e)$ ;
8.    $C.\text{insert}(u)$ ;
9.    $C.\text{insert}(v)$ ;
10.   $J = e \cup \delta(u) \cup \delta(v)$ ;
11.   $I = I / J$ ;
12. }
13. return  $C$ ;
```

VERTEX COVER PESATO

Algoritmo 4 (doubling and rounding down)

Doubling and rounding down effettuato dopo aver risolto:

- **Rilassamento lineare**
- Rilassamento lineare con odd cycle inequalities

VERTEX COVER PESATO

Algoritmo 4 (doubling and rounding down)

$$\min \sum_{u \in V} c_u x_u$$

$$x_u + x_v \geq 1 \quad \forall (u,v) \in E$$

$$x_u \geq 0 \quad \forall u \in V$$

0. Input: $G = (V, E)$
1. $x^* = \text{solver_relaxed_min_vertex_cover}(G);$
2. $w = 0^{|V|};$
3. for $(i \leftarrow 1 \text{ to } |V|)$ {
4. if $(x_i^* \geq \frac{1}{2})$ then $w_i = 1;$
5. }
6. return $w;$

VERTEX COVER PESATO

Algoritmo 4 (doubling and rounding down)

Doubling and rounding down effettuato dopo aver risolto:

- Rilassamento lineare
- **Rilassamento lineare con odd cycle inequalities**

VERTEX COVER PESATO

Algoritmo 4 (doubling and rounding down con odd cycle inequalities)

```
0. Input:  $G = (V, E)$ 
1.  $violated = true;$ 
2.  $x^* = 0;$ 
3. while( $violated$ ) {
4.    $x^* = \text{solver\_relaxed\_min\_vertex\_cover}(G);$ 
5.    $I = \{\};$  #insieme di odd cycle inequalities violate
6.    $I = \text{odd\_cycle\_violated\_by}(x^*);$ 
7.   if( $I.isEmpty()$ ) {
8.      $violated = false;$ 
9.   } else {
10.     $\text{add\_to\_solver\_constrains}(I);$ 
11.  }
12. }
13.  $w = 0^{|V|};$ 
14. for ( $i \leftarrow 1$  to  $|V|$ ) {
15.   if( $x_i^* \geq \frac{1}{2}$ ) then  $w_i = 1;$ 
16. }
17. return  $w;$ 
```

VERTEX COVER PESATO

Algoritmo 5 (basato sul duale del rilassamento lineare)

0. Input: $G = (V, E)$
1. $x = 0^{|V|}$;
2. $y = 0^{|E|}$;
3. $I = E$; #archi “scoperti”
4. while($x \neq \text{vertex_cover}(G)$) {
5. $e = I[0]$; #prendo un arco scoperto
6. $u = \text{first_vertex}(e)$;
7. $v = \text{second_vertex}(e)$;
8. $\text{increment_u} = c_u - \sum_{e \in \delta(u)} y_e$;
9. $\text{increment_v} = c_v - \sum_{e \in \delta(v)} y_e$;
10. if($\text{increment_u} \leq \text{increment_v}$) {
11. $y_e = \text{increment_u}$;
12. $x_u = 1$;
13. $I = I / \delta(u)$;

VERTEX COVER PESATO

Algoritmo 5 (basato sul duale del rilassamento lineare)

```
14.     } else {  
15.      $y_e = \text{increment\_v};$   
16.      $x_v = 1;$   
17.      $I = I / \delta(v);$   
18. }  
19. }  
20. return x;
```

ROADMAP:

1. IDEA DEL PROGETTO

2. ALGORITMI CONFRONTATI

3. METODOLOGIA

4. RISULTATI PIÙ SIGNIFICATIVI

5. CONCLUSIONI

CLASSI DI ISTANZE

- Minimum vertex cover
- Minimum weighted vertex cover

CLASSI DI ISTANZE

- **Minimum vertex cover**
- Minimum weighted vertex cover

CLASSI DI ISTANZE

- **Minimum vertex cover**
 - Numero di vertici
 - Densità del grafo
 - Grado massimo di un vertice

NUMERO DI VERTICI

Assunzione: anziché suddividere l'intervallo $\{1, \dots, N_{\max}\}$ in sotto-intervalli, si è preferito fissare dei valori «ragionevolmente distanti tra loro» che potessero fungere da rappresentanti per tali sotto-intervalli.

N_i = #vertici della classe i

$$N_i = 50 * 2^i$$

NUMERO DI VERTICI

Classi:

- 50
- 100
- 200
- 400
- 800

DENSITÀ DEL GRAFO

Concetto: è il rapporto tra il numero di archi presenti nel grafo e il numero di archi possibili.

$$d(G) = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

Classi:

- 0.2
- 0.5
- 0.8

GRADO MASSIMO DI UN VERTICE

Assunzione: anziché utilizzare il grado massimo di un vertice (che è un valore fisso), si è cercato un modo per parametrizzarlo in funzione del numero dei vertici e degli archi del grafo.

Osservazione: non è tanto di interesse il valore in sé del grado massimo di un vertice, quanto lo «sbilanciamento» del grafo e la disposizione dei vertici di grado maggiore rispetto a quelli di grado inferiore.

GRADO MASSIMO DI UN VERTICE

Rapporto grado massimo-medio: è il rapporto tra il grado massimo tra tutti i vertici e la media del grado di tutti i vertici del grafo.

Classi:

- 1 (omogeneità assoluta)
- Tra 1.2 e 1.6 (leggero sbilanciamento)
- Maggiore di 2 (sbilanciamento evidente)

CLASSI DI ISTANZE

- Minimum vertex cover
- **Minimum weighted vertex cover**

CLASSI DI ISTANZE

- **Minimum weighted vertex cover**
 - Numero di vertici
 - Densità del grafo
 - Variabilità dei pesi dei vertici
 - Correlazione tra grado e peso di un vertice

VARIABILITÀ DEI PESI

Variabile aleatoria: si è impiegata una variabile aleatoria uniforme nell'intervallo $[a,b]$ con $a = 1$.

Varianza: per aumentare la varianza $(\frac{(b-a)^2}{12})$ si modifica il secondo estremo dell'intervallo.

Classi (valori di b):

- 10
- 1000
- 10000

CORRELAZIONE TRA GRADO E PESO DI UN VERTICE

Classi (in base al coefficiente di Spearman):

- -1 (correlazione monotona decrescente)
- 0 (nessuna correlazione)
- 1 (correlazione monotona crescente)

OTTIMIZZAZIONI NEL CODICE

Il codice implementato **non mappa perfettamente** lo pseudocodice fornito, in quanto sono state fatte alcune considerazioni in termini di ottimizzazione per diminuire il tempo di calcolo e impiegare meno memoria.

OTTIMIZZAZIONI NEL CODICE

Esempio:

0. Input: $G = (V, E)$ #grafo
1. $C = \{\}$; #cover
2. $I = E$; #archi "scoperti"
3. for v in V {
4. $C.insert(v)$;
5. $I = I / \delta(v)$;
6. if ($I.isEmpty$) then return C ;
7. }

```
def vertex_cover_non_pesato_1(G):  
    C = set()  
    archi_da_coprire = G.number_of_edges()  
    for v in G.nodes():  
        C.add(v)  
        for u in G.neighbors(v):  
            if u not in C:  
                archi_da_coprire -= 1  
        if archi_da_coprire == 0:  
            return C  
    return C
```

OTTIMIZZAZIONI NEL CODICE

Esempio:

0. Input: $G = (V, E)$ #grafo

1. $C = \{\}$; #cover

```
def vertex_cover_non_pesato_1(G):  
    C = set()
```

Sebbene l'idea sia la stessa, rimuovere dall'insieme degli archi la stella uscente da un vertice ogni volta che questo viene inserito nella cover è subottimale rispetto a decrementare semplicemente il contatore degli archi.

6. if (I.isEmpty) then return C;

7. }

```
    if archi_da_coprire == 0:  
        return C  
    return C
```

OTTIMIZZAZIONI NEL CODICE

Esempio:

Nella generazione delle odd cycle inequalities

```
if x_vals[u] + x_vals[v] + x_vals[w] < 1.999:
```

Sebbene matematicamente andrebbe inserito il < 2 , si è osservato che la tolleranza del solver è di 6 cifre decimali, per evitare loop dovuti a quantità trascurabili si è deciso di considerarne solo 3.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato

```
def applica_configuration_model(n, gradi_float):  
    gradi_int = np.round(gradi_float).astype(int)  
    gradi_int = np.clip(gradi_int, 0, n - 1)  
    if np.sum(gradi_int) % 2 != 0:  
        idx = np.random.randint(0, n)  
        gradi_int[idx] += 1 if gradi_int[idx] < n - 1 else -1  
    G_multi = nx.configuration_model(gradi_int)  
    G = nx.Graph(G_multi)  
    G.remove_edges_from(nx.selfloop_edges(G))  
    return G
```

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

- `np.round(gradi_float).astype(int)`
 - Arrotonda gli eventuali gradi decimali.
- `np.clip(gradi_int, 0, n - 1)`
 - Forza il grado di ogni nodo ad assumere un valore compreso tra 0 e $|V| - 1$.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
if np.sum(gradi_int) % 2 != 0:  
    idx = np.random.randint(0, n)  
    gradi_int[idx] += 1 if gradi_int[idx] < n - 1 else -1
```

Se si sono generati dei gradi incompatibili con l'Handshaking Lemma si prende un vertice qualsiasi e se ne modifica il grado di 1.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
G_multi = nx.configuration_model(gradi_int)
G = nx.Graph(G_multi)
G.remove_edges_from(nx.selfloop_edges(G))
```

Si costruisce il multigrafo in base ai gradi generati, si rimuovono gli archi duplicati e i self loop.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato

```
def genera_singola_istanza(n, densita_target, classe_eterogeneita):  
    k_medio_atteso = densita_target * (n - 1)  
    if classe_eterogeneita == 0:  
        k = int(np.round(k_medio_atteso))  
        if (n * k) % 2 != 0:  
            k += 1  
        return nx.random_regular_graph(k, n)  
    elif classe_eterogeneita == 1:  
        std_dev = k_medio_atteso * 0.14  
        while True:  
            gradi_float = np.random.normal(loc=k_medio_atteso, scale=std_dev, size=n)  
            H = np.max(gradi_float) / np.mean(gradi_float)  
            if 1.2 <= H <= 1.6:  
                return applica_configuration_model(n, gradi_float)
```

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato

```
elif classe_eterogeneita == 2:
    while True:
        sigma = 1.0
        mu = np.log(k_medio_atteso) - (sigma ** 2 / 2)
        gradi_float = np.random.lognormal(mean=mu, sigma=sigma, size=n)
        H = np.max(gradi_float) / np.mean(gradi_float)
        if H > 2.0:
            return applica_configuration_model(n, gradi_float)
    else:
        raise ValueError(f"Classe {classe_eterogeneita} non valida. Usa 0, 1 o 2.")
```

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

- $k_medio_atteso = densita_target * (n - 1)$
 - Se la densità è $\frac{2|E|}{|V|(|V|-1)}$
 - La moltiplicazione per $|V|-1$ restituisce $\frac{2|E|}{|V|}$
 - Per l'Handshaking Lemma la somma dei gradi di tutti i vertici è esattamente $2|E|$, pertanto il rapporto ottenuto è il grado medio.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
if classe_eterogeneita == 0:  
    k = int(np.round(k_medio_atteso))  
    if (n * k) % 2 != 0:  
        k += 1  
    return nx.random_regular_graph(k, n)
```

Se è si sta considerando un rapporto grado massimo/grado medio di 1 (classe 0) allora si usa la primitiva che genera un grafo di n vertici, tutti con stesso grado, pari al valore medio trovato. Da notare l'arrotondamento e la modifica del grado se si viola l'Handshaking Lemma.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
elif classe_eterogeneita == 1:
    std_dev = k_medio_atteso * 0.14
    while True:
        gradi_float = np.random.normal(loc=k_medio_atteso, scale=std_dev, size=n)
        H = np.max(gradi_float) / np.mean(gradi_float)
        if 1.2 <= H <= 1.6:
            return applica_configuration_model(n, gradi_float)
```

Si utilizza una normale perché è una variabile aleatoria la cui varianza non si ottiene in modo matematico a partire dalla media (si può modificare a piacimento mantenendo la media fissa).

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

- $\text{std_dev} = \text{k_medio_atteso} * 0.14$
 - $H = \frac{\text{deg_max}}{\text{deg_medio}}$
 - Per una normale: $\text{val_max} \approx \mu + 3 \sigma$
 - Quindi $H \approx \frac{\mu + 3 \sigma}{\mu}$
 - Si è imposto $H \in [1.2, 1.6]$, si considera $H \approx \frac{1.2+1.6}{2} = 1.4$

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

- $\text{std_dev} = k_{\text{medio_atteso}} * 0.14$
 - $H \approx \frac{\mu + 3 \sigma}{\mu} = 1.4$
 - Si ottiene $\sigma = 0.1333 \mu \approx 0.14 \mu$

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
elif classe_eterogeneita == 1:
    std_dev = k_medio_atteso * 0.14
    while True:
        gradi_float = np.random.normal(loc=k_medio_atteso, scale=std_dev, size=n)
        H = np.max(gradi_float) / np.mean(gradi_float)
        if 1.2 <= H <= 1.6:
            return applica_configuration_model(n, gradi_float)
```

H è il rapporto tra grado massimo e grado medio, ci si assicura che sia effettivamente nel range desiderato prima di restituire il grafo, altrimenti si ritenta.

GENERAZIONE DELLE ISTANZE

Vertex cover non pesato – Istruzioni che vale la pena commentare:

```
sigma = 1.0  
mu = np.log(k_medio_atteso) - (sigma ** 2 / 2)  
gradi_float = np.random.lognormal(mean=mu, sigma=sigma, size=n)
```

Si usa una lognormale perché ha una heavy tail e permette di «sbilanciare» il grafo (alternando vertici di grado molto alto a vertici di grado molto basso). $\sigma = 1$ è stato ritenuto un valore coerente per evitare la formazione di pochissimi vertici con altissimo grado ma senza generare valori troppo simili a quelli di una normale.

GENERAZIONE DELLE ISTANZE

Vertex cover pesato

```
def genera_singola_istanza_pesata(n, densita_target, variabilita_peso, correlazione_spearman):  
    G = nx.erdos_renyi_graph(n, p=densita_target)  
    pesi_generati = np.random.randint(1, variabilita_peso + 1, size=n)  
    if correlazione_spearman == 0:  
        np.random.shuffle(pesi_generati)  
        mapping_pesi = {nodo: int(peso) for nodo, peso in zip(G.nodes(), pesi_generati)}  
    else:  
        nodi_ordinati = sorted(G.nodes(), key=lambda x: G.degree(x))  
        pesi_ordinati = sorted(pesi_generati)  
        if correlazione_spearman == -1:  
            pesi_ordinati = pesi_ordinati[::-1]  
        mapping_pesi = {nodo: int(peso) for nodo, peso in zip(nodi_ordinati, pesi_ordinati)}  
    nx.set_node_attributes(G, mapping_pesi, 'peso')  
    return G
```

GENERAZIONE DELLE ISTANZE

Vertex cover pesato – Istruzioni che vale la pena commentare:

- `nx.erdos_renyi_graph(n, p=densita_target)`
 - Genera un grafo dato il numero di vertici e la densità; usa la densità come probabilità per generare gli archi.
- `np.random.randint(1, variabilita_peso + 1, size=n)`
 - Estrae valori da una variabile aleatoria uniforme; è stato necessario aumentare di 1 il valore del secondo parametro perché la funzione genera nativamente valori nell'intervallo `[primo_parametro, secondo_parametro)`.

SVOLGIMENTO - 1

Classi di istanze da testare: incrociando i criteri di partizionamento precedentemente definiti (densità, eterogeneità del grado, ecc.), si ottengono per combinazione tutte le classi di test oggetto del progetto.

Solver scelto: Gurobi con licenza accademica (ritenuto il migliore per problemi relativi ai grafi).

SVOLGIMENTO - 2

Risultati effettivamente riportati: di seguito solo i risultati più interessanti osservati. Per l'elenco completo dei risultati:

https://github.com/gianpaolo-pestilli/amod_project.git

Risoluzione del timeout: se il solver va in timeout prima di trovare l'ottimo, si seleziona il miglior lower bound trovato fino a quel momento e si usa per calcolare il rapporto di approssimazione.

SVOLGIMENTO - 3

Crash del solver nel calcolo dell'ottimo: se Gurobi riporta errori su una specifica istanza, quella viene ignorata poiché non si ha a disposizione alcun lower bound con cui fare confronti.

Algoritmi valutati in base a:

- Rapporto di approssimazione medio
- Tempo di esecuzione medio

RUN EFFETTUATI

- **Minimum vertex cover**
- Minimum weighted vertex cover

RUN EFFETTUATI

- **Minimum vertex cover**
 - $5*3*3 = 45$ classi
 - Per ogni istanza il solver viene utilizzato in 3 algoritmi
 - Tempo limite del solver = 10 secondi
 - Numero di istanze per classe = 10
 - **Tempo di esecuzione massimo = 3.75 ore**

RUN EFFETTUATI

- Minimum vertex cover
- **Minimum weighted vertex cover**

RUN EFFETTUATI

- **Minimum weighted vertex cover**
 - $5*3*3*3 = 135$ classi
 - Per ogni istanza il solver viene utilizzato in 3 algoritmi
 - Tempo limite del solver = 10 secondi
 - Numero di istanze per classe = 10
 - **Tempo di esecuzione massimo = 11.25 ore**

RUN EFFETTUATI

- Minimum we

- $5*3*3*3 =$

- Per ogni ist

- Tempo limi

- Numero di

- Tempo di e



algoritmi

RUN REALMENTE EFFETTUATI

- **Minimum weighted vertex cover**
 - $5*3*3*3 = 135$ classi
 - Per ogni istanza il solver viene utilizzato in 3 algoritmi
 - Tempo limite del solver
 - 8 secondi per PLI e PLI rilassato
 - 10 secondi per PLI rilassato + ODI

RUN REALMENTE EFFETTUATI

- **Minimum weighted vertex cover**
 - Numero di istanze per classe = 5
 - **Tempo di esecuzione massimo = 4.875 ore**

RUN REALMENTE EFFETTUATI

Conseguenze: il numero ridotto di istanze potrebbe produrre risultati meno significativi.

ROADMAP:

1. IDEA DEL PROGETTO

2. ALGORITMI CONFRONTATI

3. METODOLOGIA

4. RISULTATI PIÙ SIGNIFICATIVI

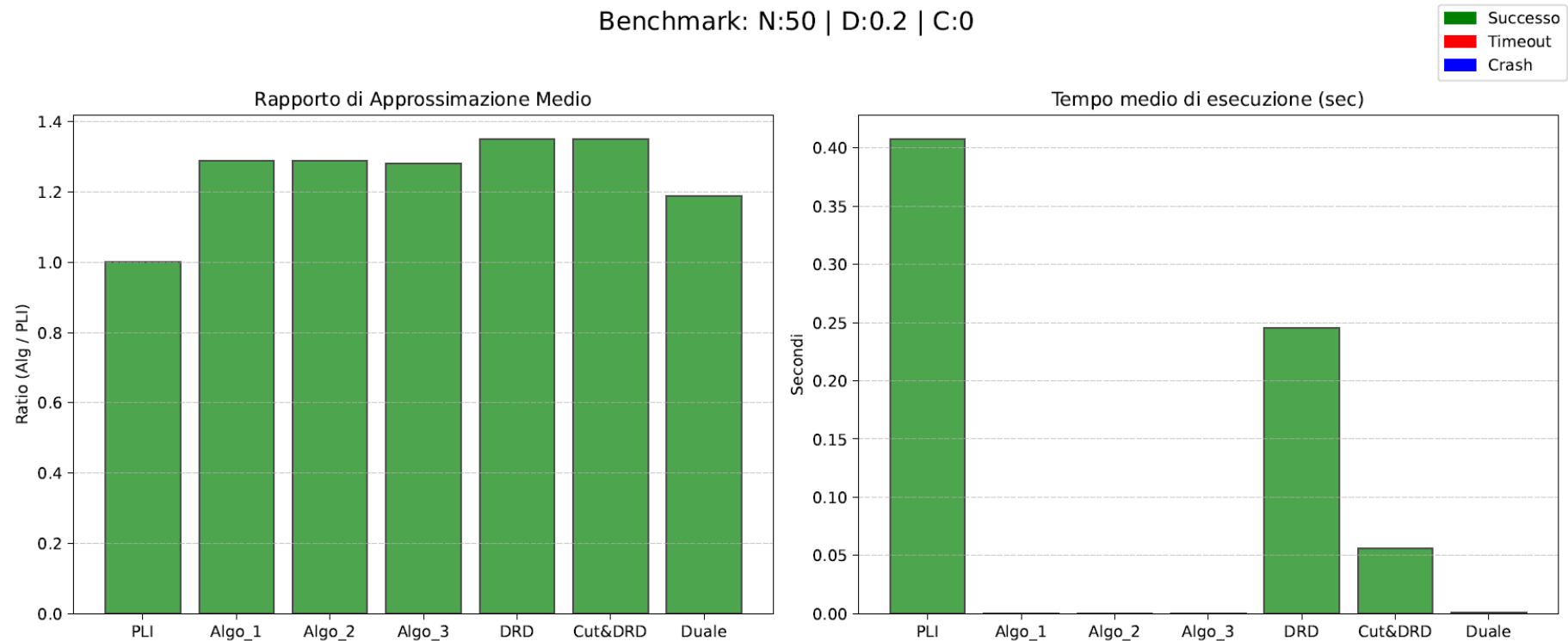
5. CONCLUSIONI

RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- **Miglior algoritmo**
 - Algoritmo basato sul duale
 - Rapporto di approssimazione medio sempre < 1.25
 - Eccelle per classi con eterogeneità 2
 - Sbilanciamento evidente

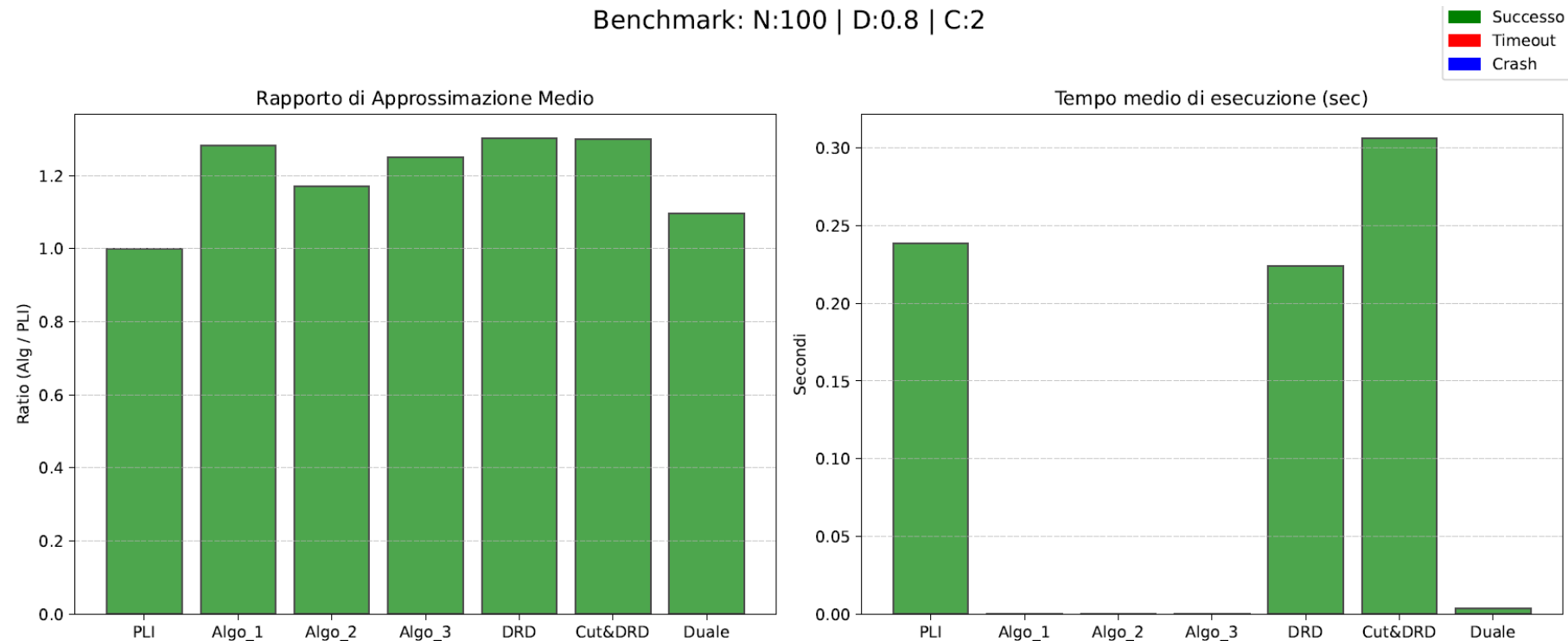
RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- **Miglior algoritmo**



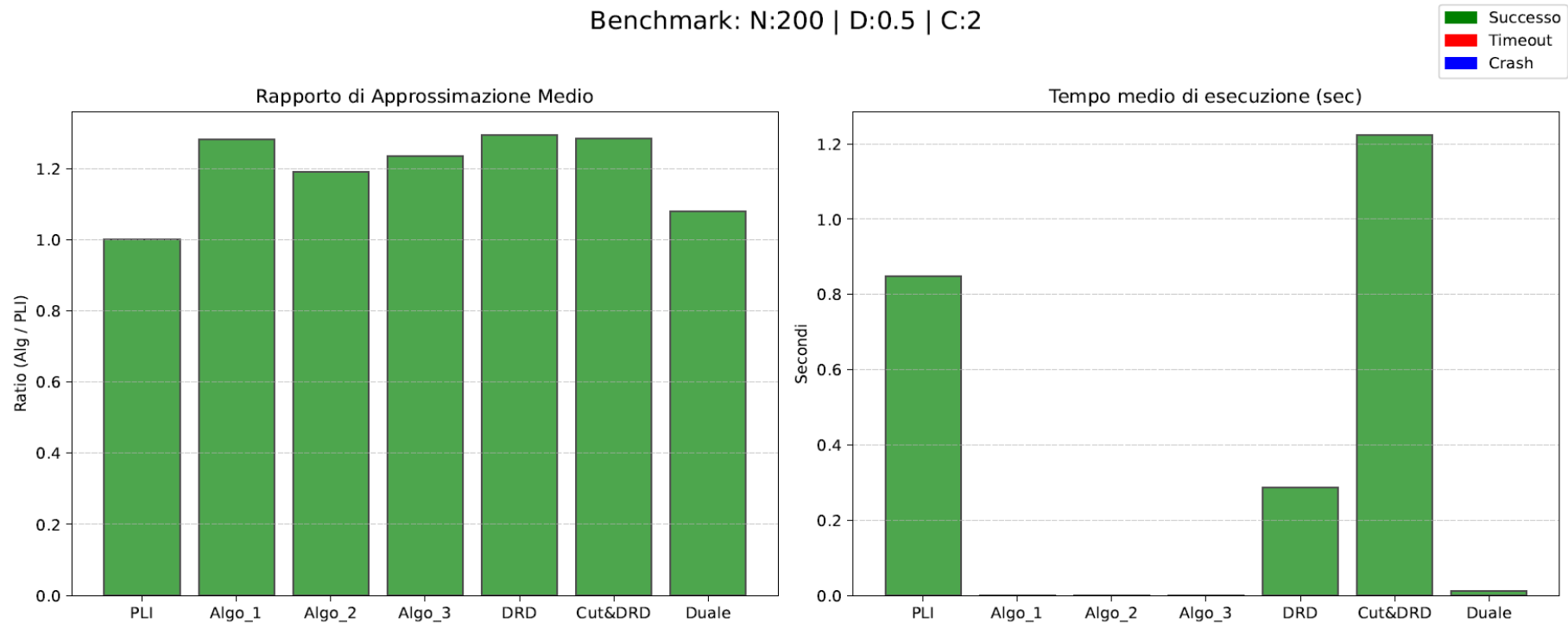
RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- **Miglior algoritmo**



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

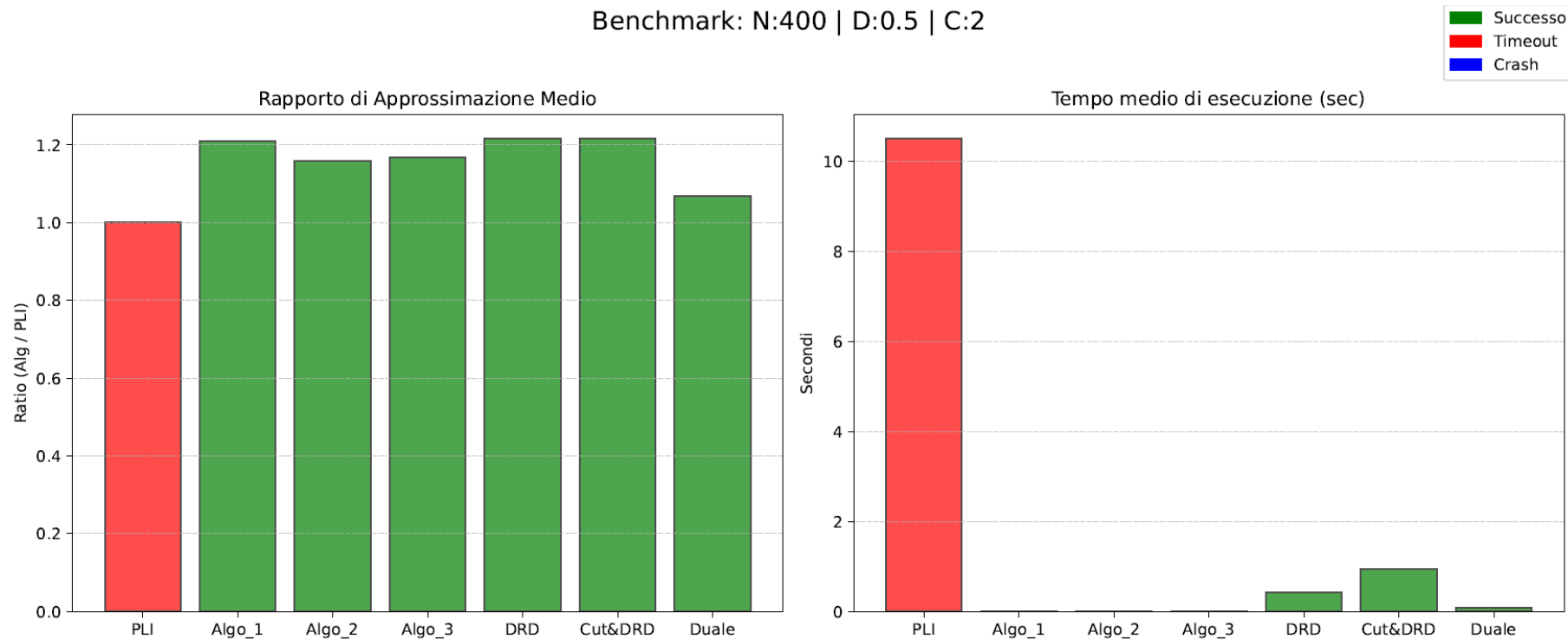
- **Miglior algoritmo**



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Miglior algoritmo (anche in caso di timeout)

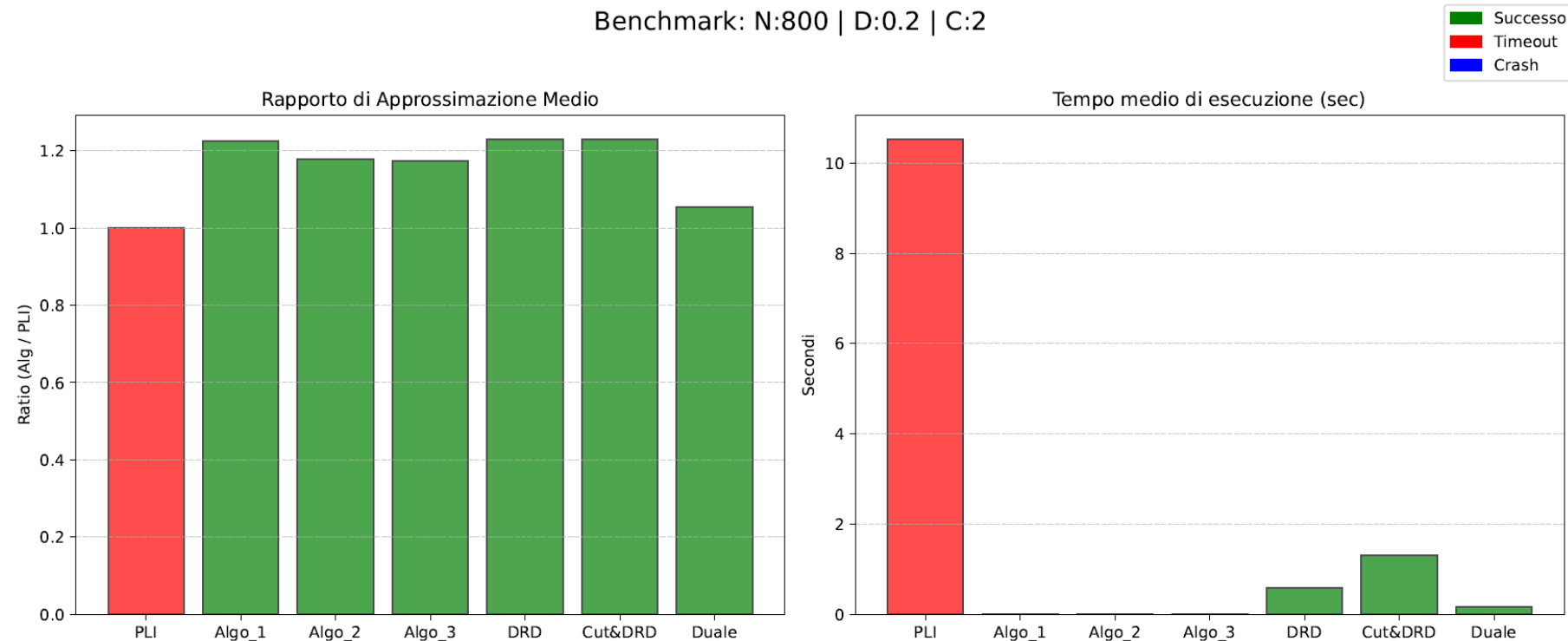
Benchmark: N:400 | D:0.5 | C:2



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

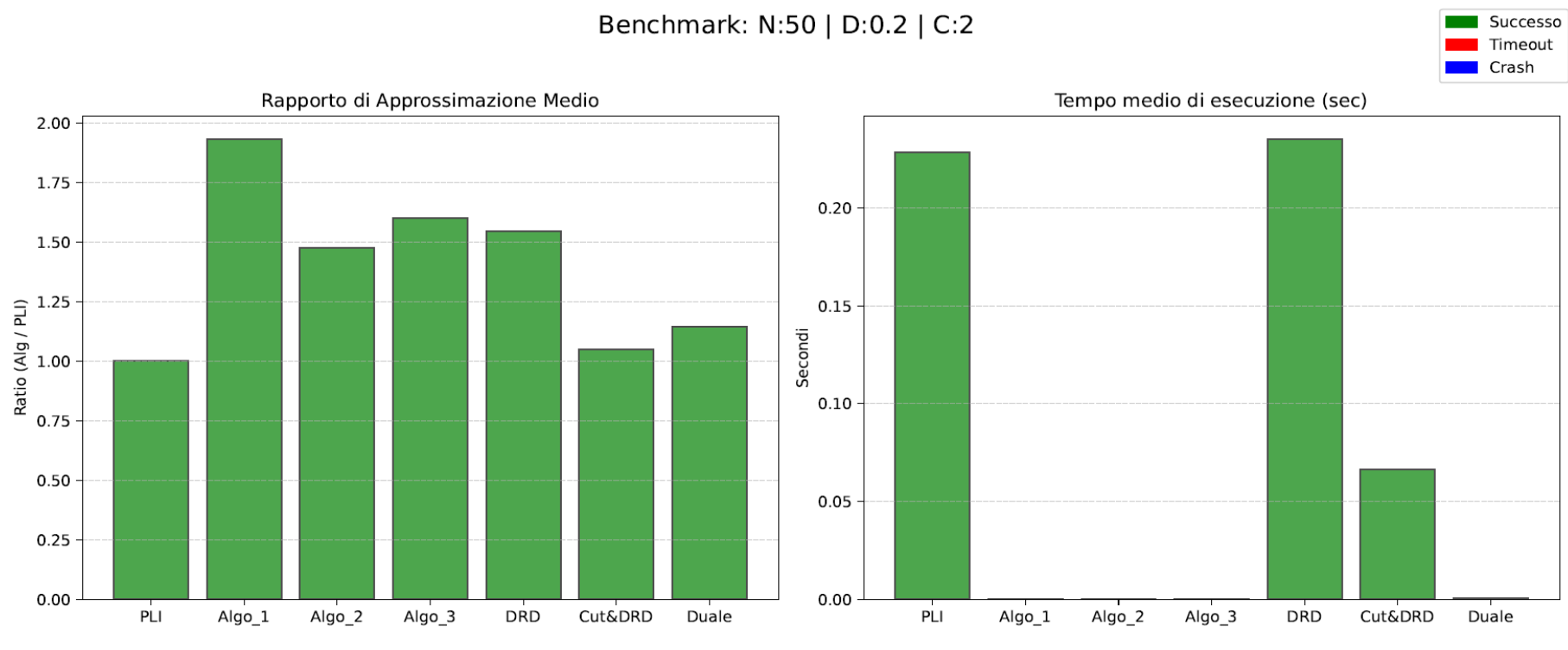
- Miglior algoritmo (anche in caso di timeout)

Benchmark: N:800 | D:0.2 | C:2



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

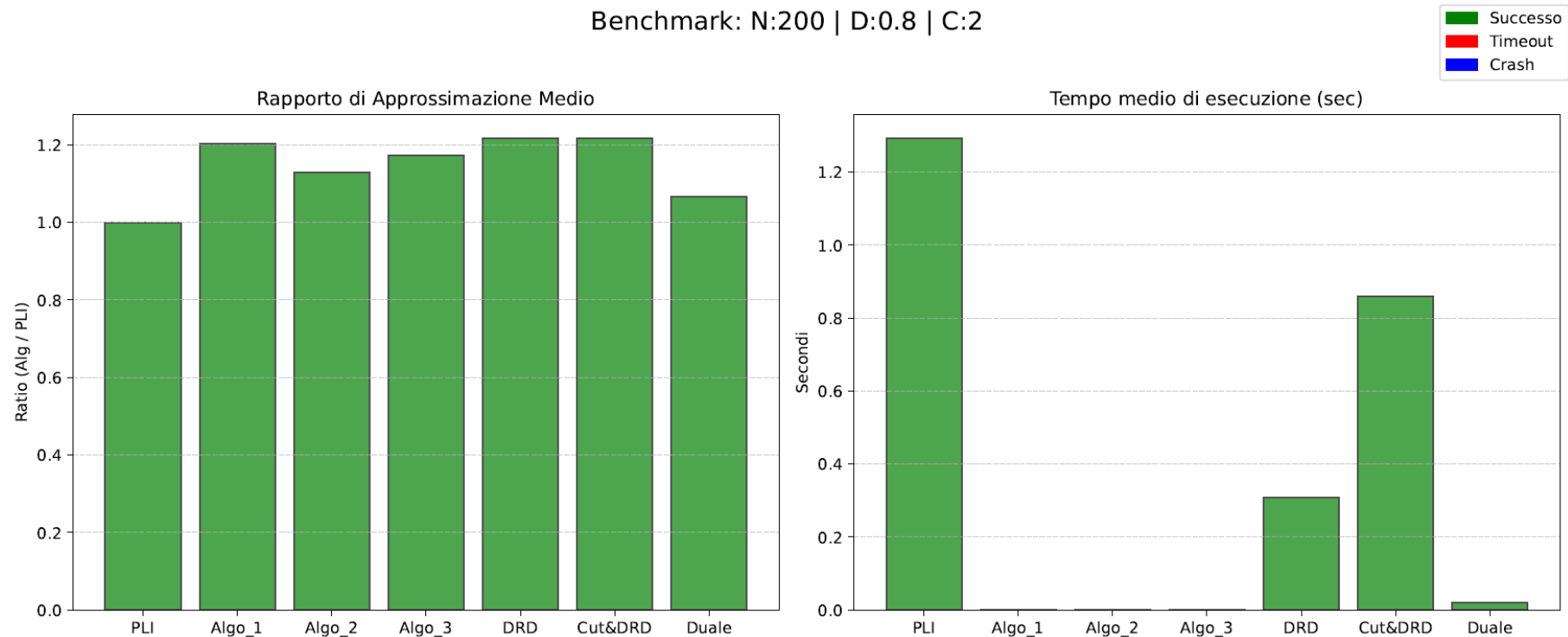
- **Algoritmo 1 (parte da un vertice qualsiasi): peggiore?**



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- **Algoritmo 1 (parte da un vertice qualsiasi): peggiore?**

Benchmark: N:200 | D:0.8 | C:2



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

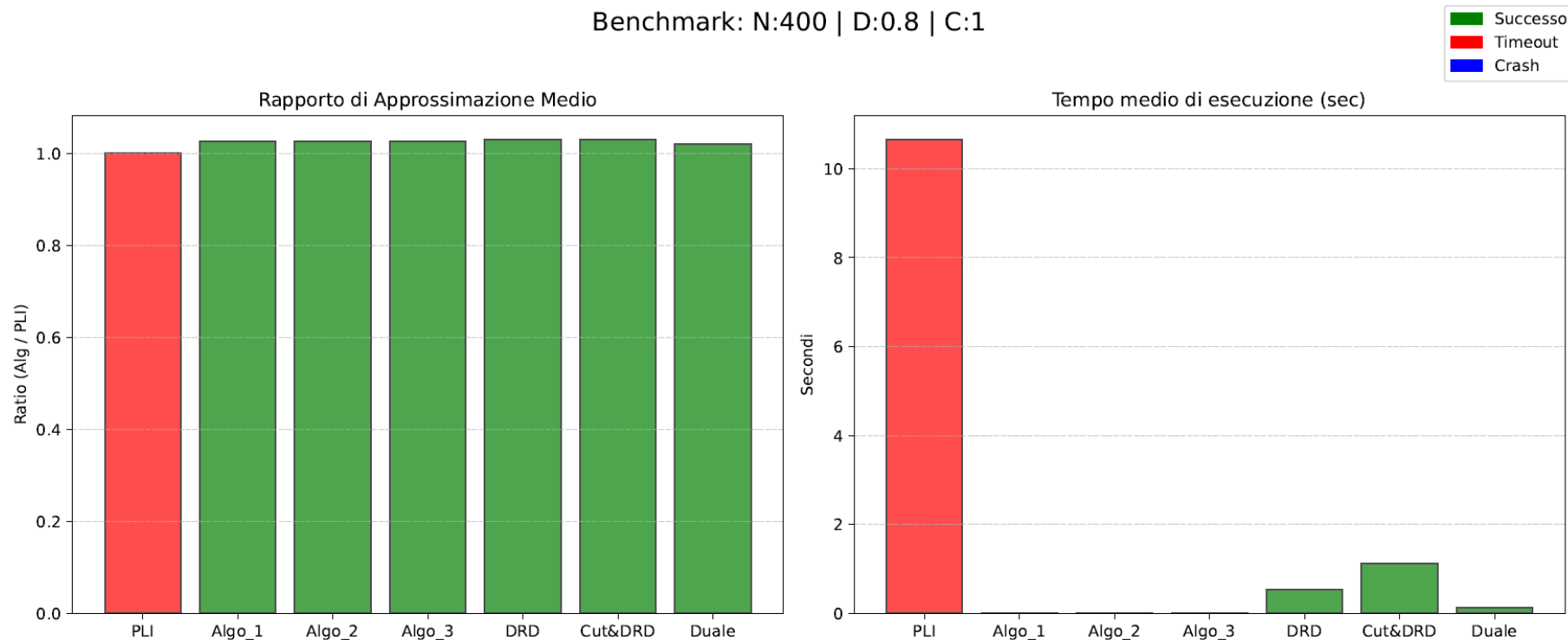
- Algoritmo 1 (parte da un vertice qualsiasi): peggiore?



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Grafo denso, con molti vertici e «mediamente eterogeneo»

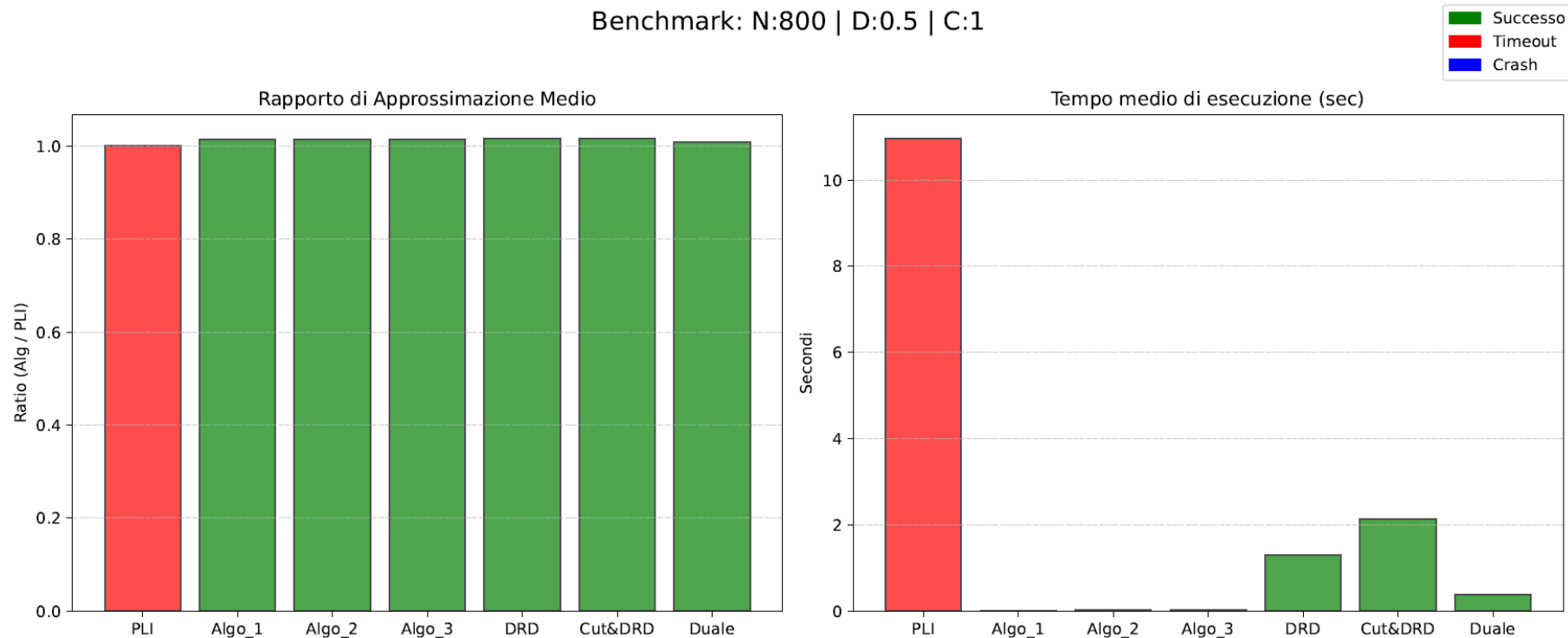
Benchmark: N:400 | D:0.8 | C:1



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Grafo denso, con molti vertici e «mediamente eterogeneo»

Benchmark: N:800 | D:0.5 | C:1



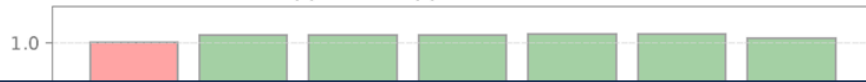
RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Grafo denso, con molti vertici e «mediamente eterogeneo»

Benchmark: N:800 | D:0.5 | C:1

Successo
Timeout
Crash

Rapporto di Approssimazione Medio



Tempo medio di esecuzione (sec)

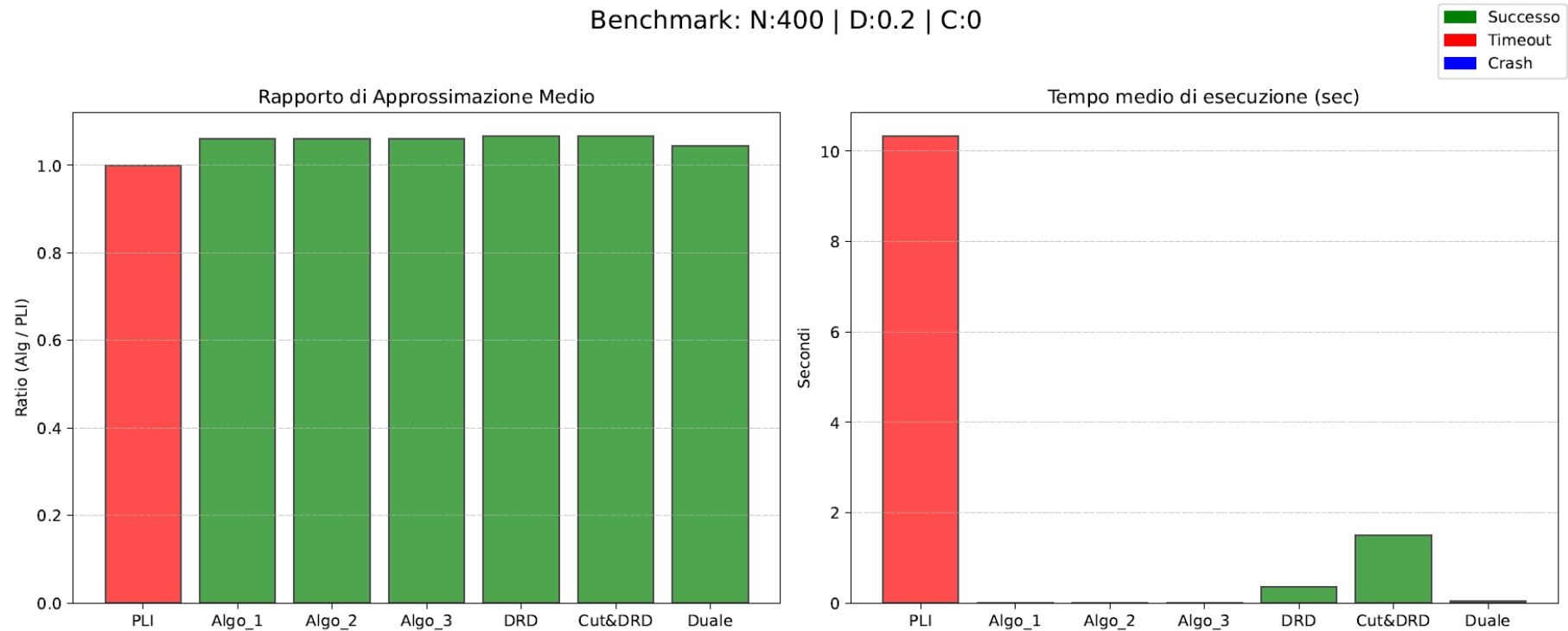


Rapporti di approssimazione simili, sono da preferire gli algoritmi che non utilizzano solver



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Timeout del Solver



RISULTATI OTTENUTI – VERTEX COVER NON PESATO

- Timeout del Solver

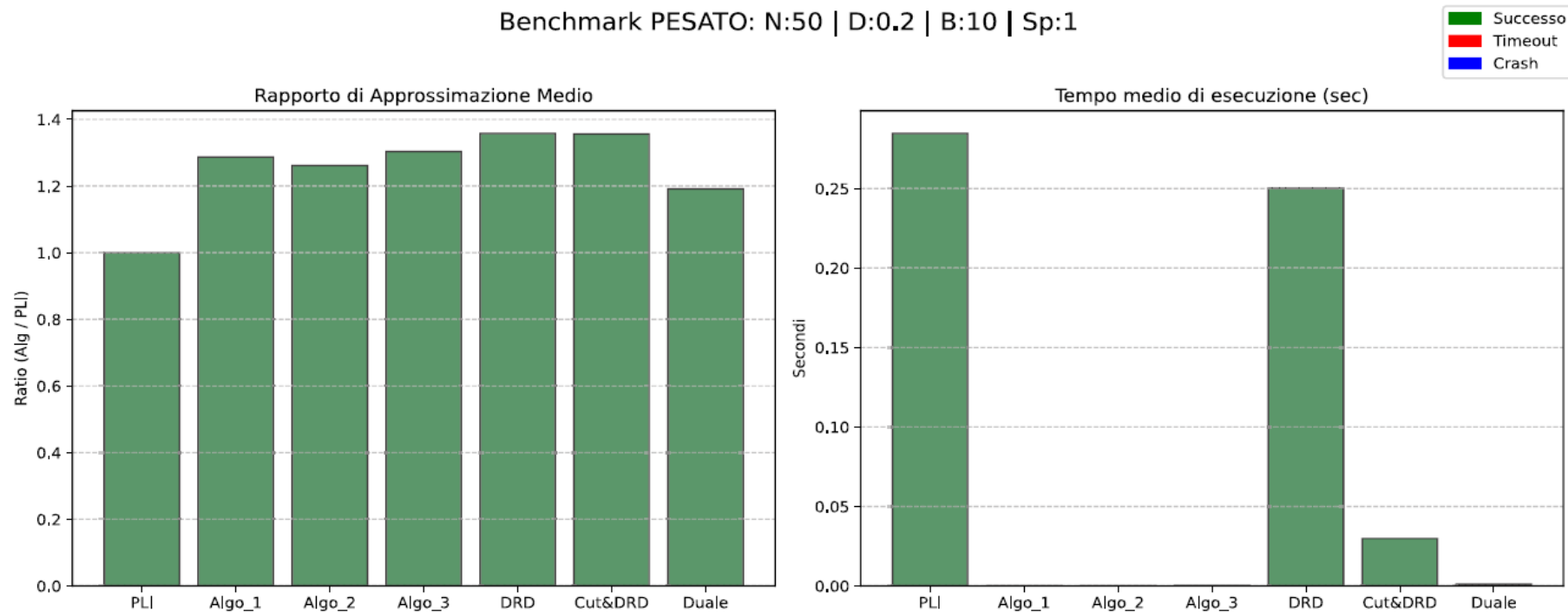


RISULTATI OTTENUTI – VERTEX COVER PESATO

- **Miglior algoritmo**
 - Algoritmo basato sul duale
 - Rapporto di approssimazione medio sempre < 1.25
 - Eccelle per classi con pochi vertici e preferibilmente con coefficiente di Spearman non nullo

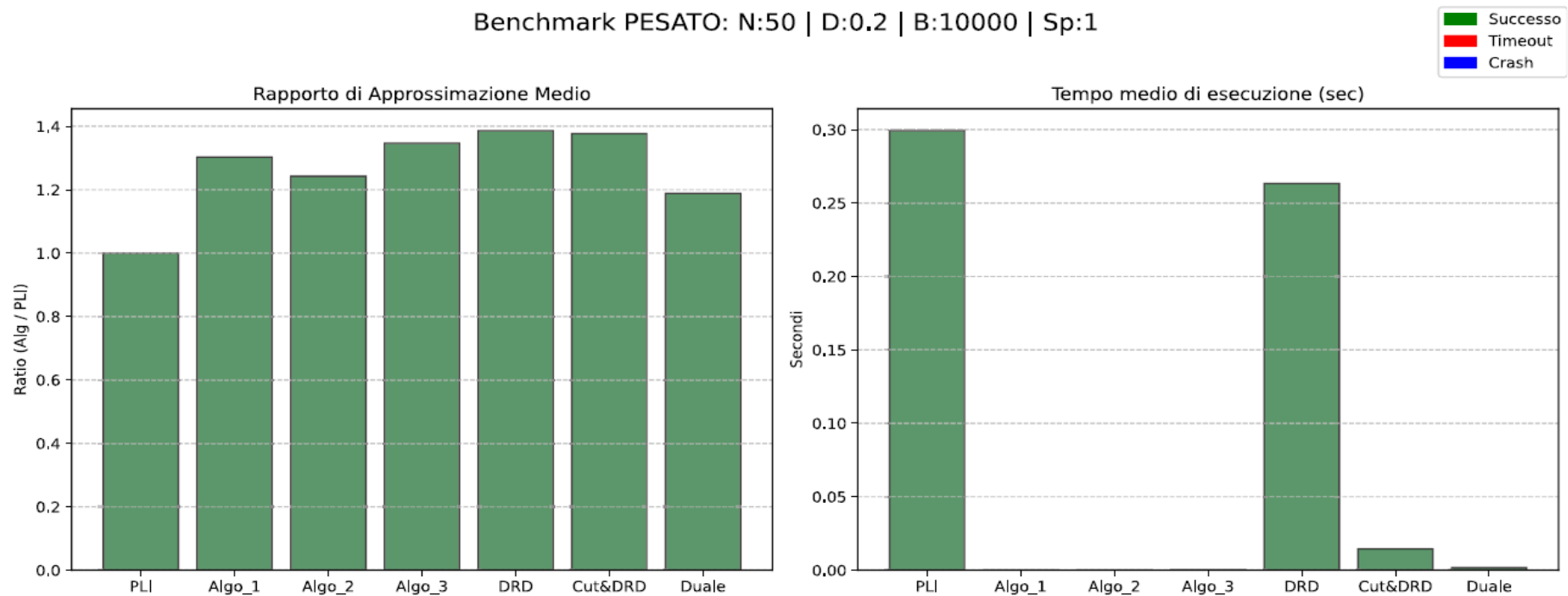
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



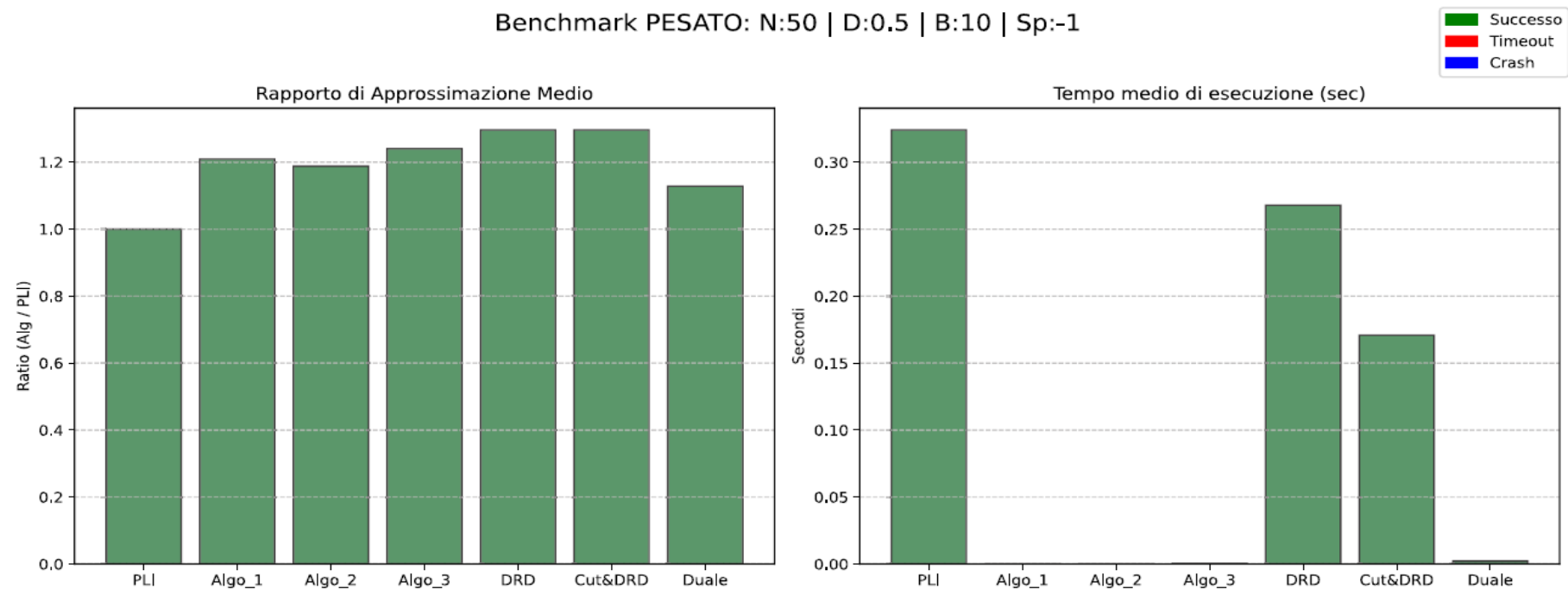
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



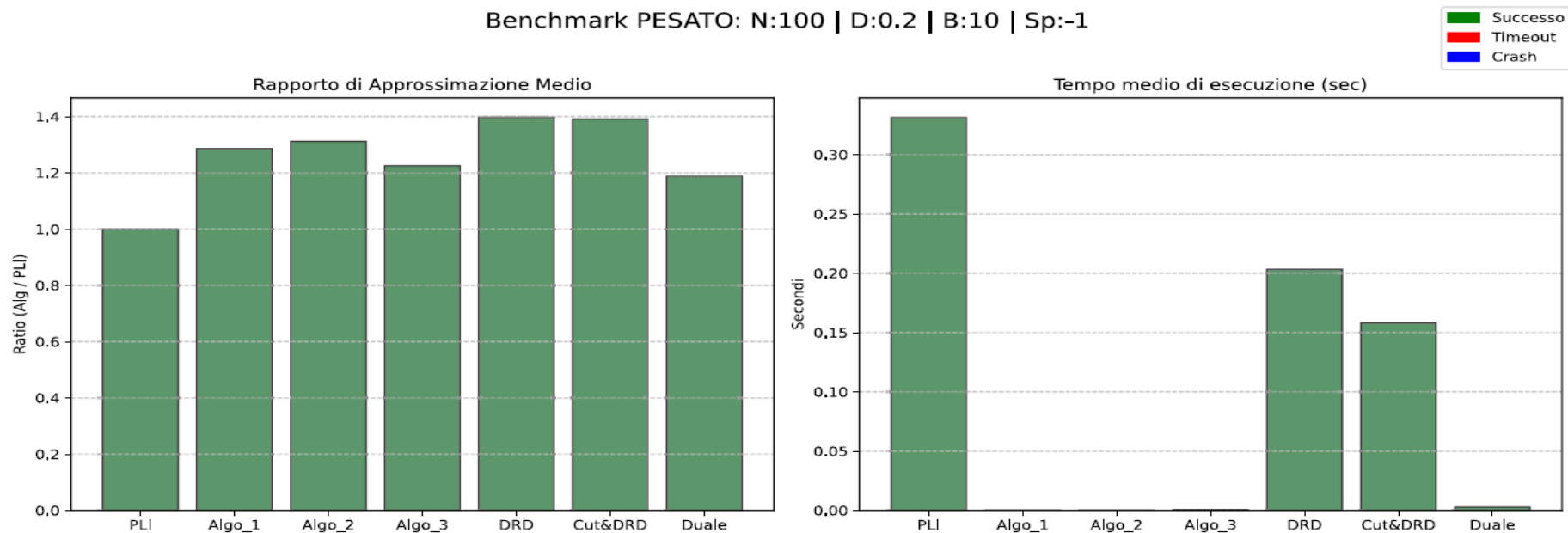
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



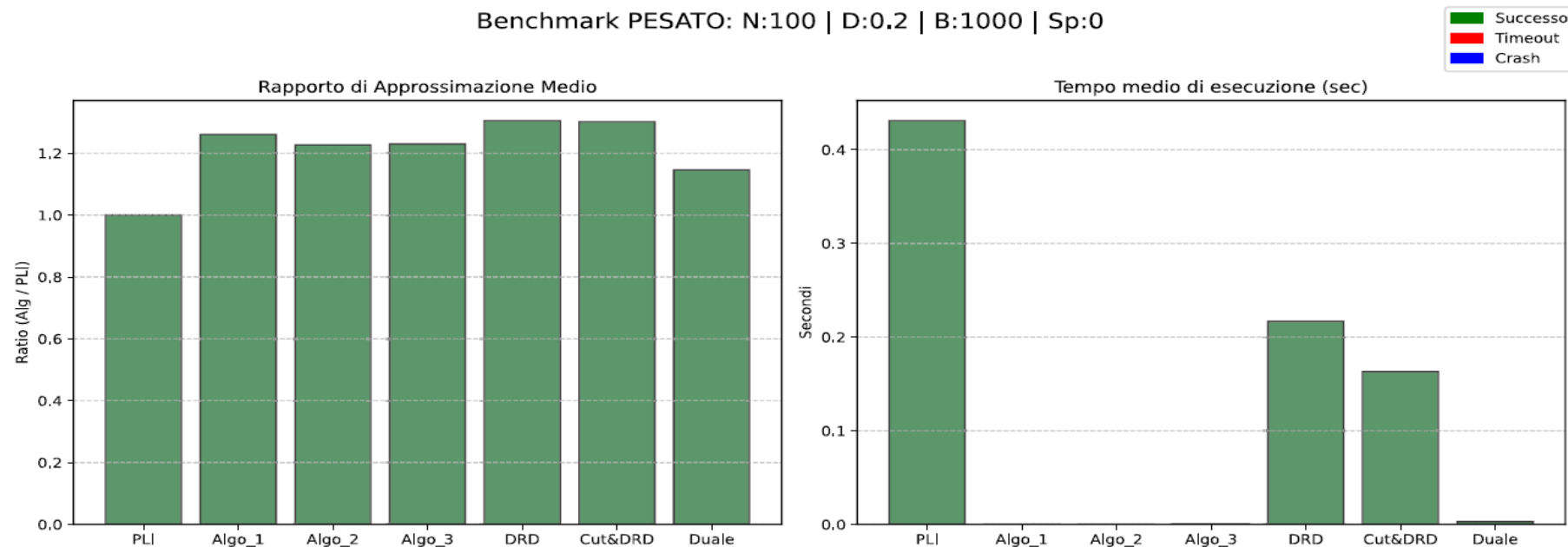
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



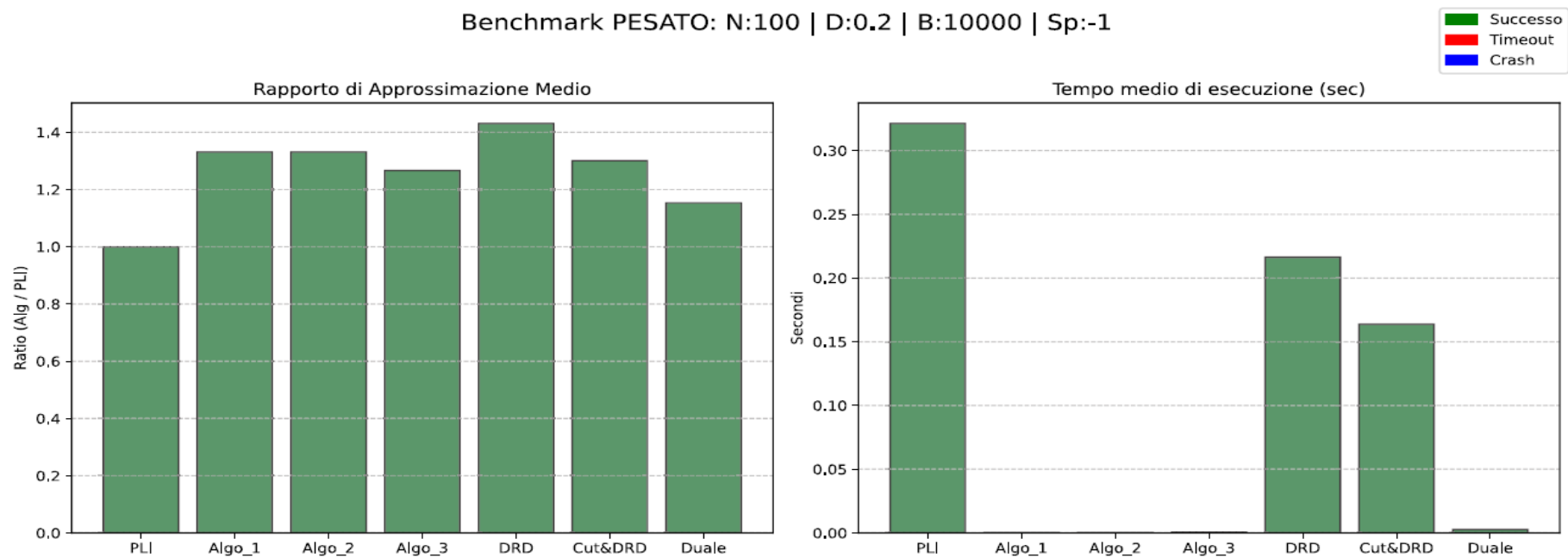
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



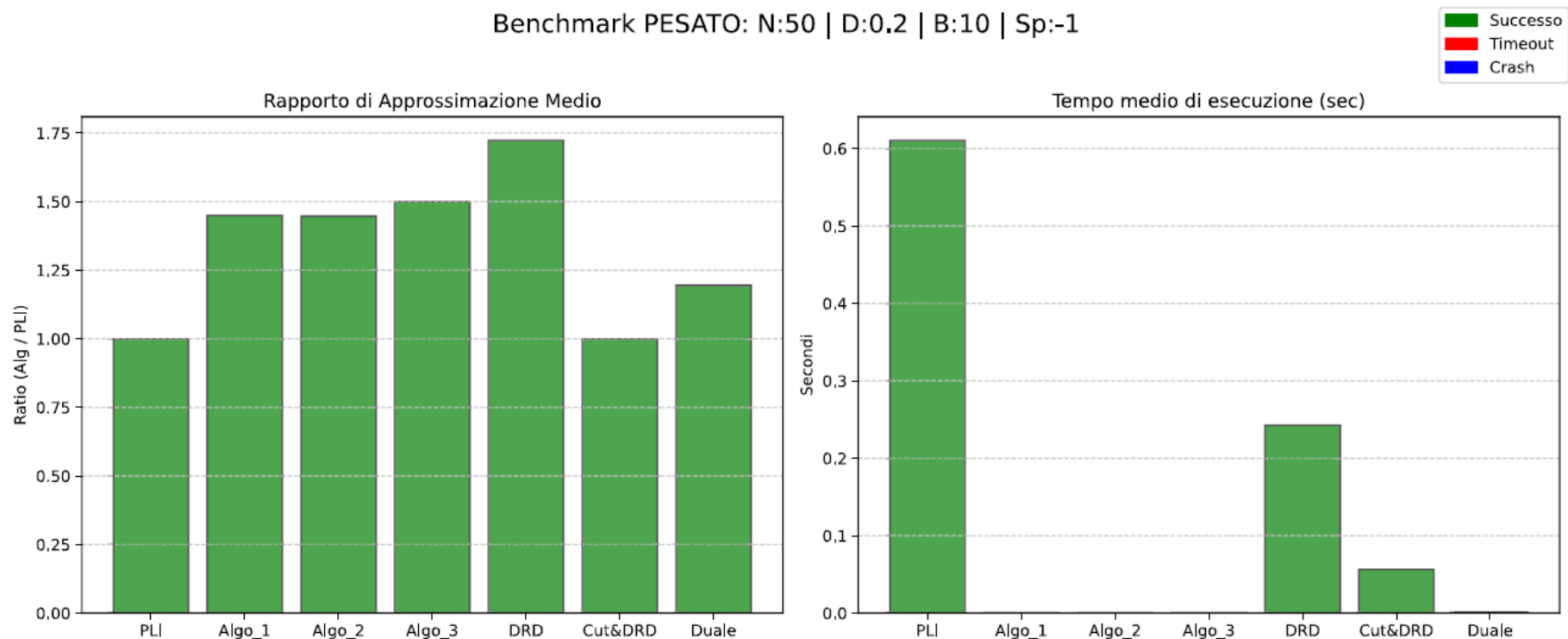
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo



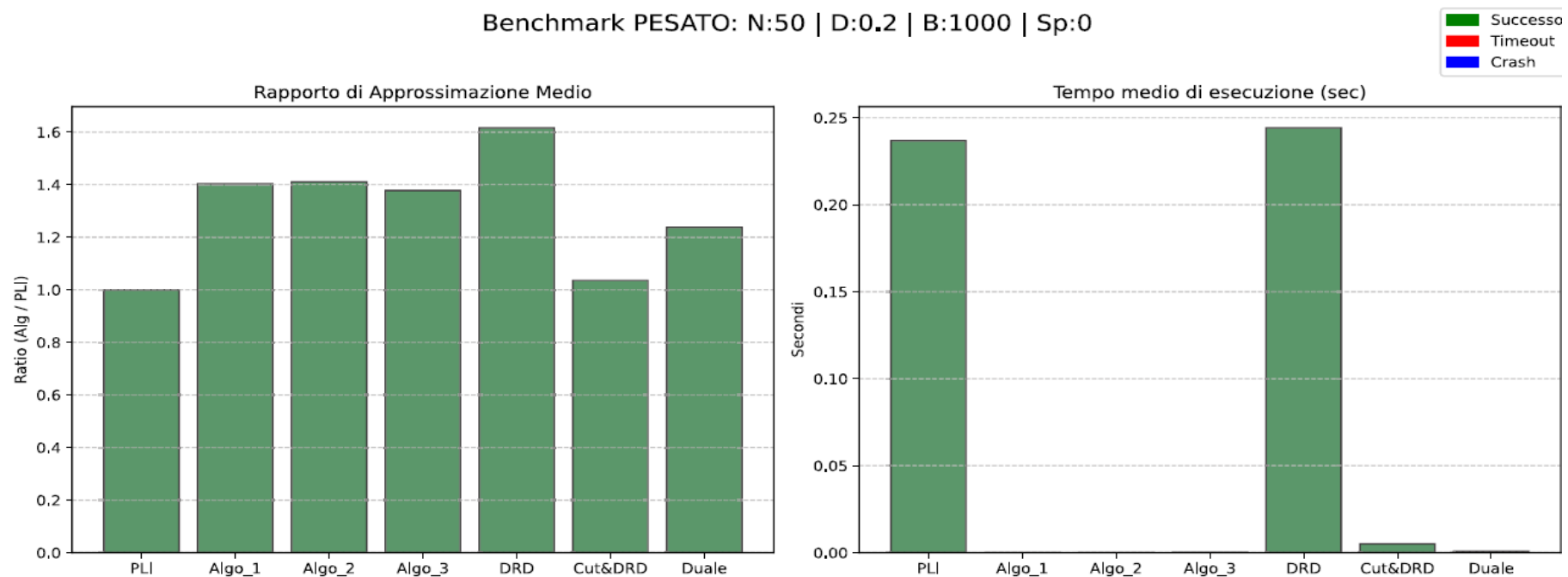
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo (quando non lo è)



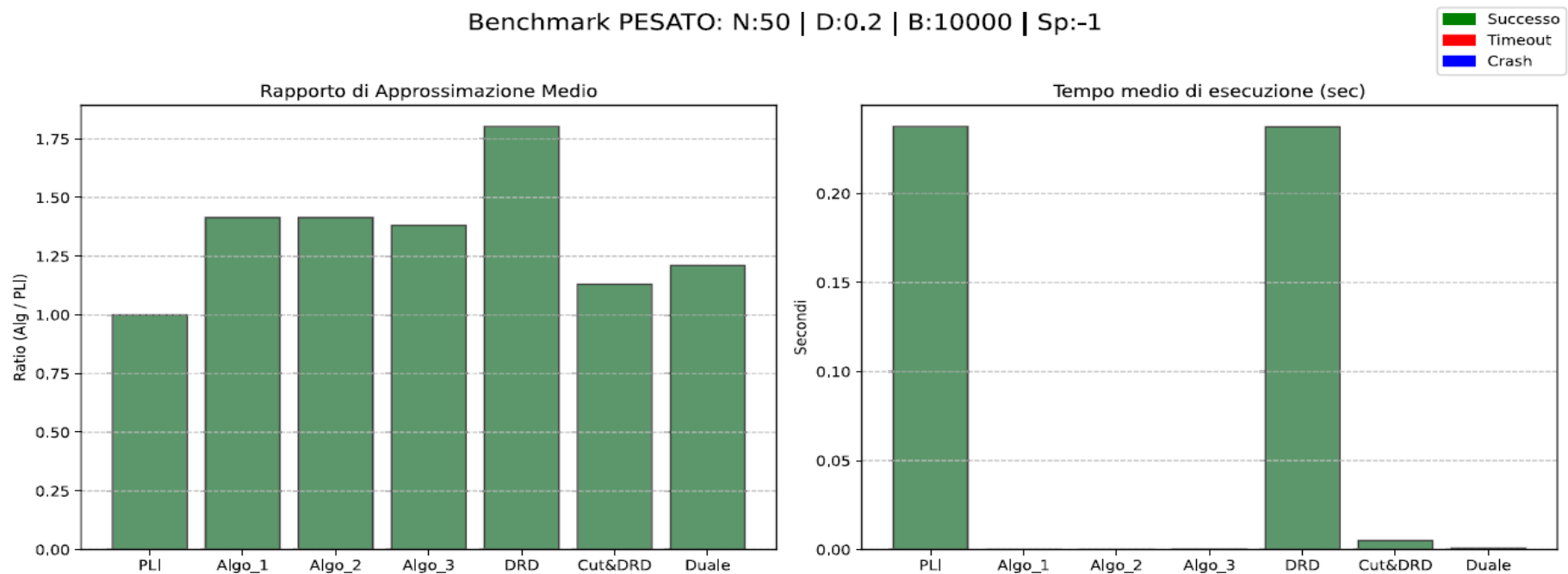
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo (quando non lo è)



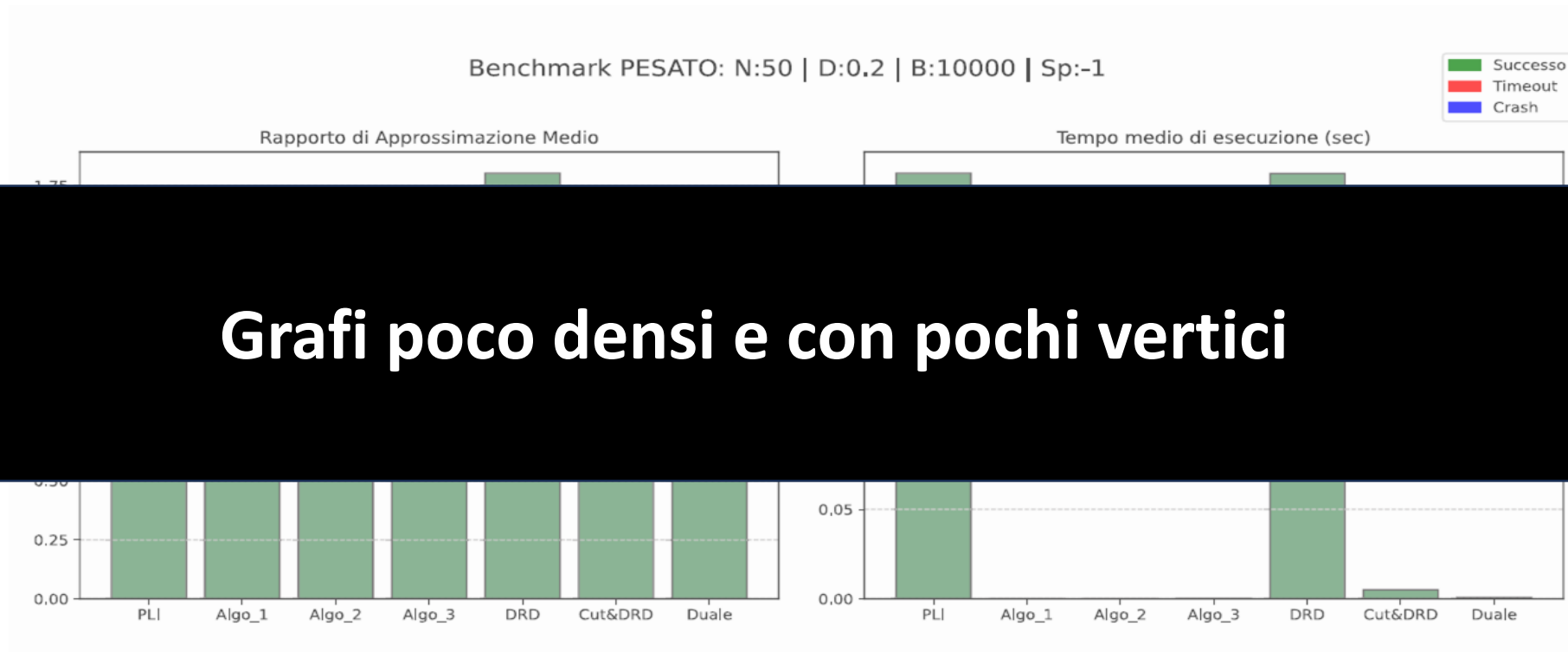
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo (quando non lo è)



RISULTATI OTTENUTI – VERTEX COVER PESATO

- Miglior algoritmo (quando non lo è)



- **RISULTATI OTTENUTI – VERTEX COVER PESATO**
- **Peggior algoritmo**
 - Algoritmo basato sul doubling and rounding down
 - Rapporto di approssimazione medio sempre < 1.75
 - La scelta di una soglia leggermente inferiore a 0.5 per inserire un vertice in cover potrebbe aver reso l'algoritmo troppo «generoso»
 - Soffre eccessivamente le istanze con alta variabilità del peso dei vertici

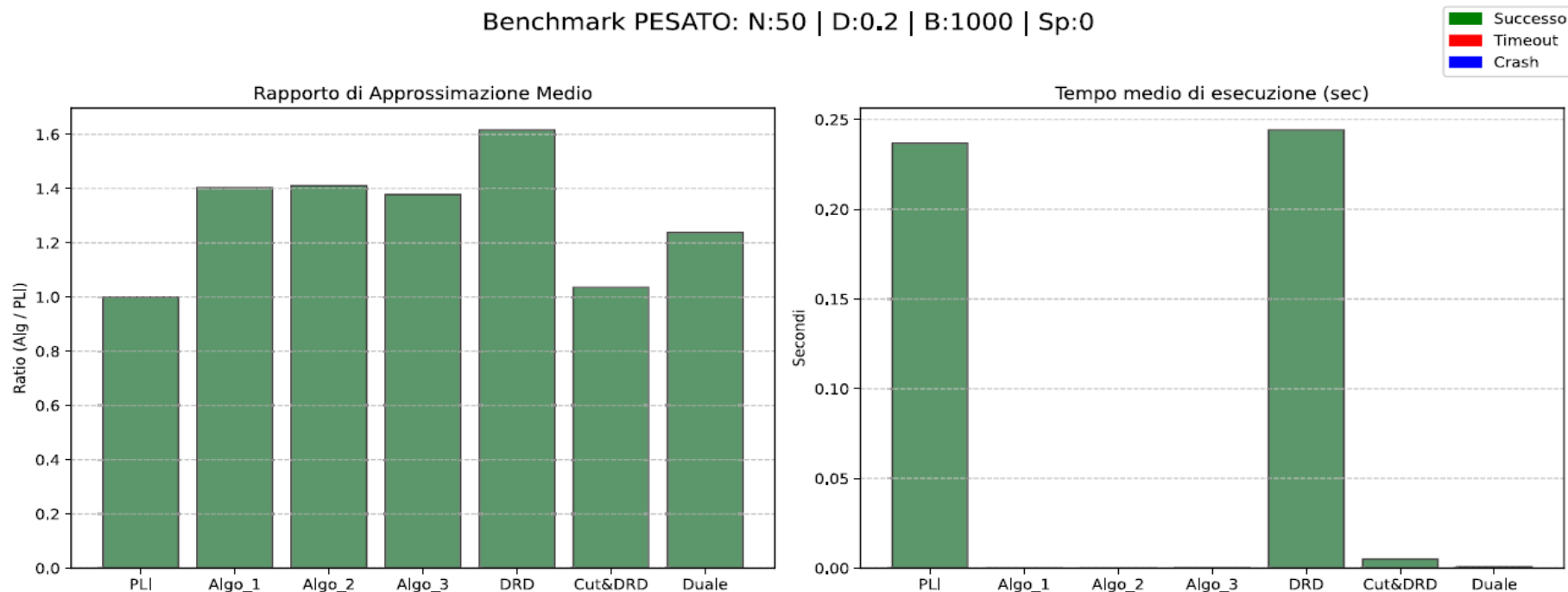
- **RISULTATI OTTENUTI – VERTEX COVER PESATO**
- Peggior algoritmo
 - Algoritmo basato sul doubling and rounding down

**Si è provato ad alzare la soglia ma sono state ottenute
soluzioni non ammissibili**

troppo «generoso»

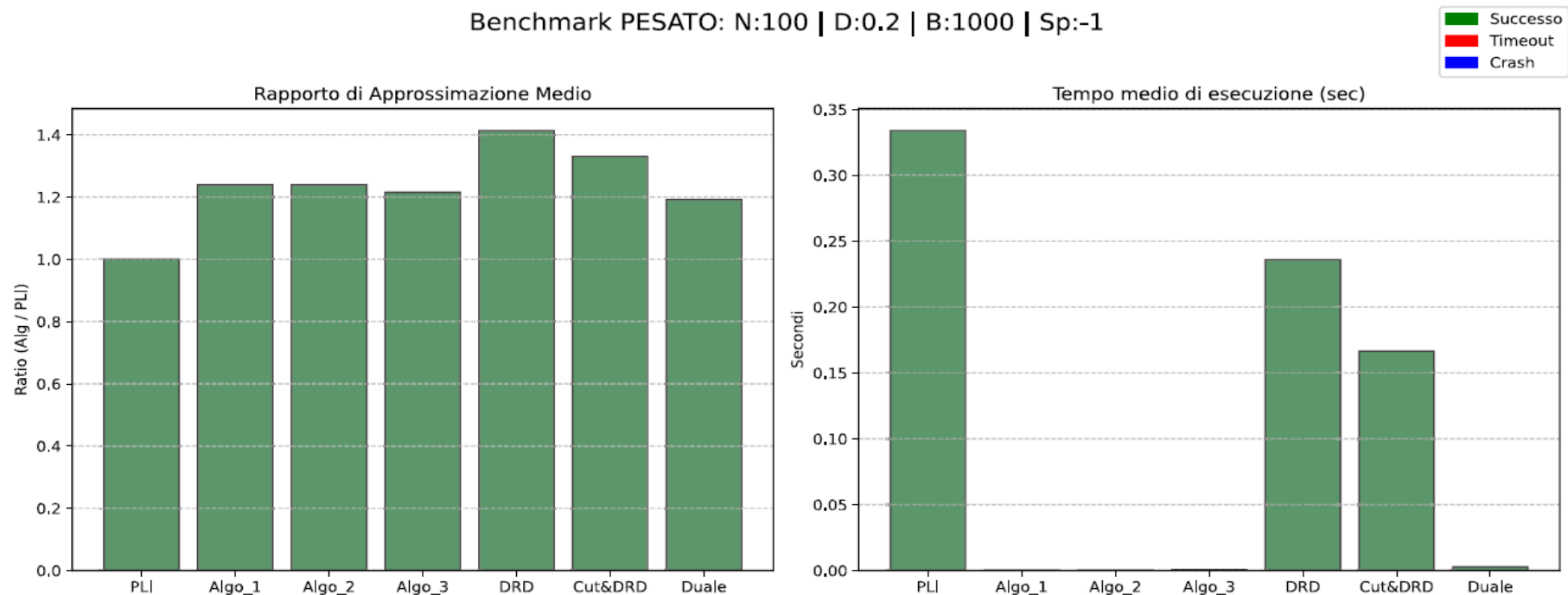
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Peggior algoritmo



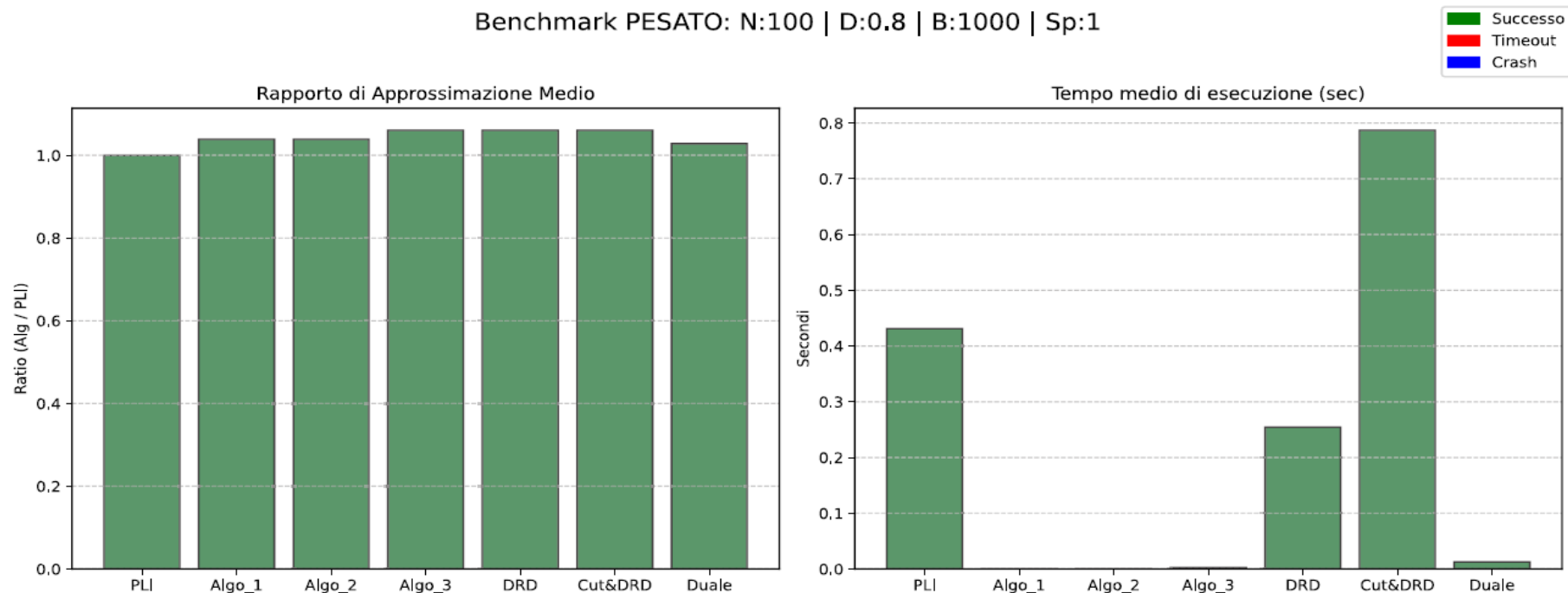
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Peggior algoritmo



RISULTATI OTTENUTI – VERTEX COVER PESATO

- Peggior algoritmo (quando non lo è)



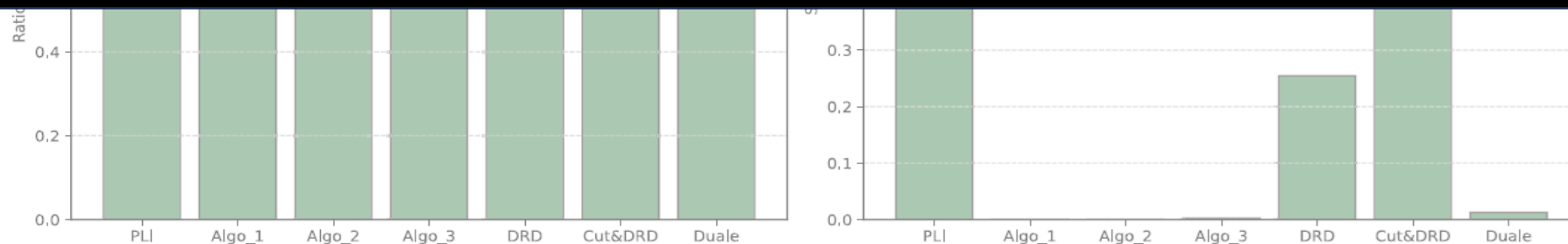
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Peggior algoritmo (quando non lo è)

Benchmark PESATO: N:100 | D:0.8 | B:1000 | Sp:1

Successo
Timeout
Crash

L'elevata variabilità dei pesi mette in particolare difficoltà gli algoritmi basati su doubling and rounding down



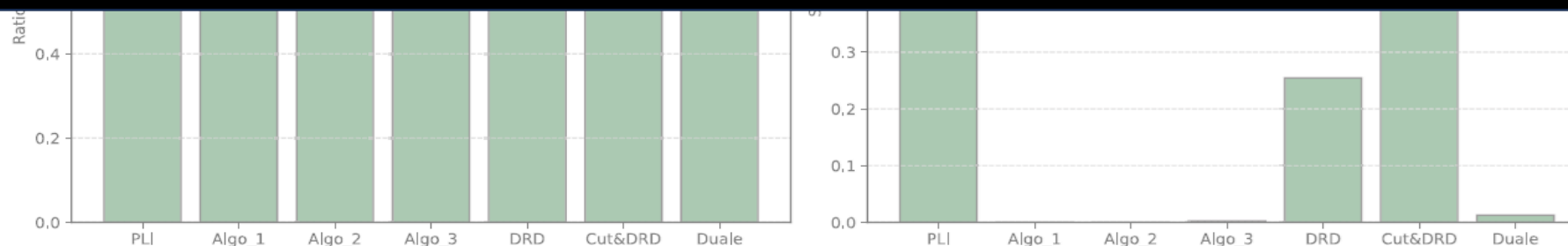
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Peggior algoritmo (quando non lo è)

Benchmark PESATO: N:100 | D:0.8 | B:1000 | Sp:1

Successo
Timeout
Crash

Grafi molto densi e con variabilità dei pesi molto alta hanno comunque messo in crisi tutti gli algoritmi

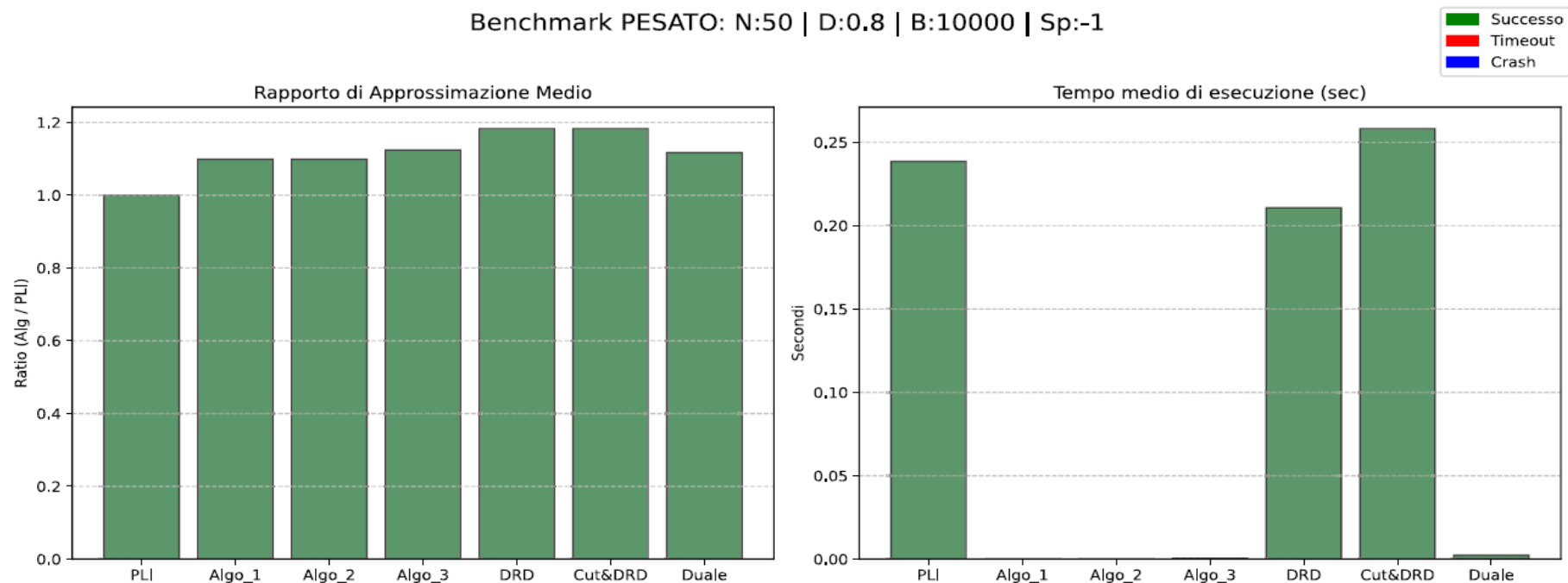


RISULTATI OTTENUTI – VERTEX COVER PESATO

- **Algoritmo 1 (parte dal vertice che pesa meno): ha risentito della correlazione?**
 - Ci si aspettavano performance migliori quando il vertice con peso minore coincideva con il vertice di grado maggiore.
 - Non è stato peggiore degli altri algoritmi

RISULTATI OTTENUTI – VERTEX COVER PESATO

- **Algoritmo 1 (parte dal vertice che pesa meno):**

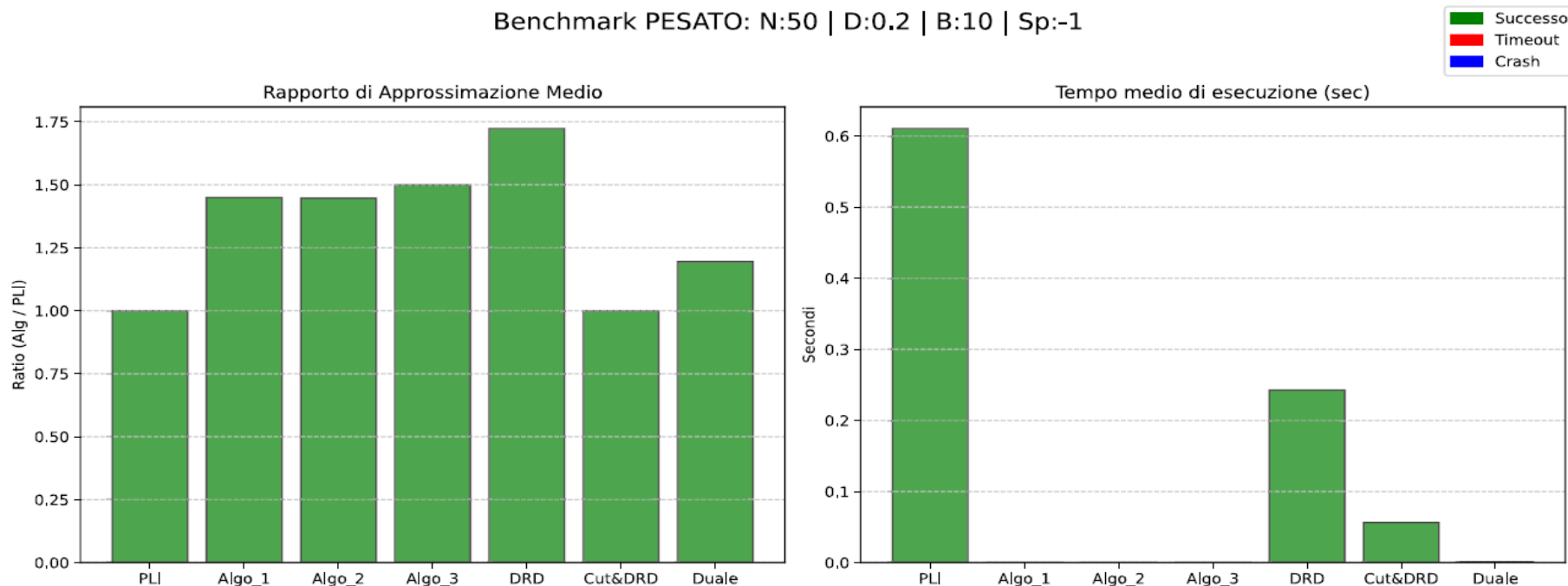


RISULTATI OTTENUTI – VERTEX COVER PESATO

- **Algoritmo 2 (parte dal vertice meno denso): ha risentito della correlazione?**
 - Non si comporta mai peggio dell'algoritmo 1 tranne quando ci sono grafi con pochi vertici e il coefficiente di Spearman è -1: l'algoritmo 1 è troppo favorito.

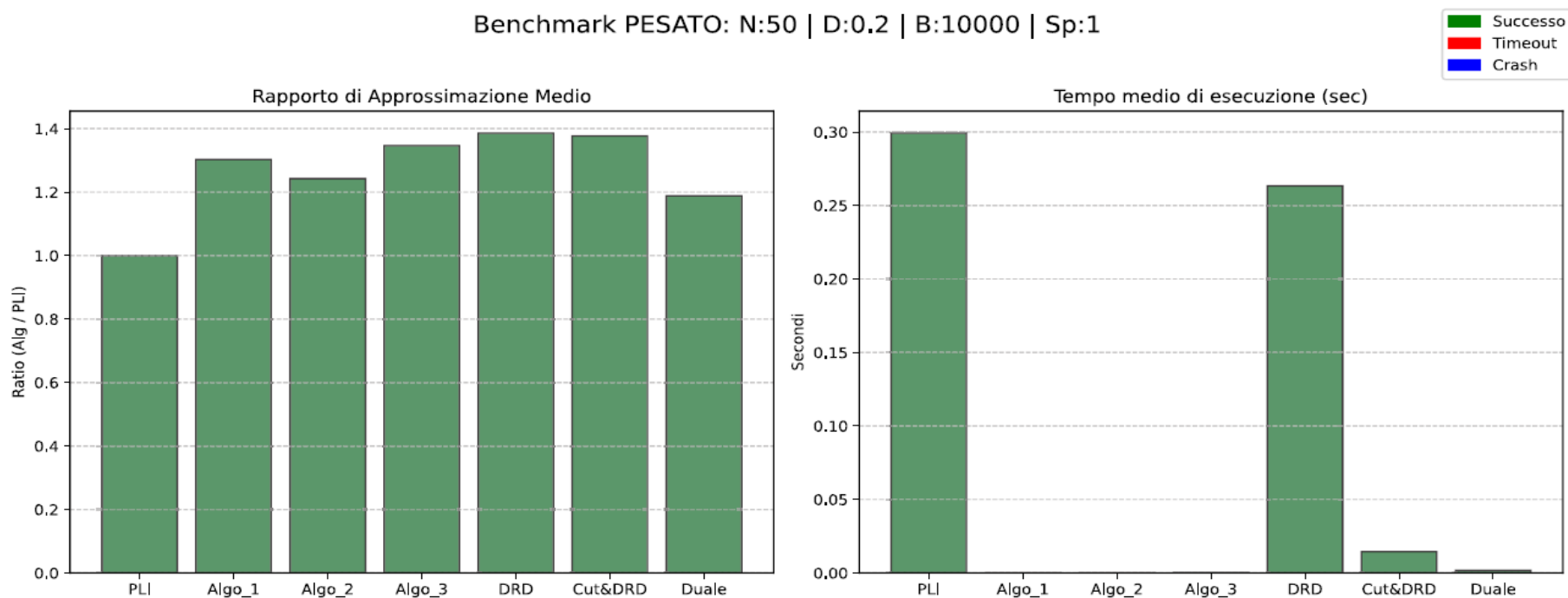
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Algoritmo 2 (parte dal vertice meno denso): rispetto ad Algoritmo 1?



RISULTATI OTTENUTI – VERTEX COVER PESATO

- Algoritmo 2 (parte dal vertice meno denso): rispetto ad Algoritmo 1?

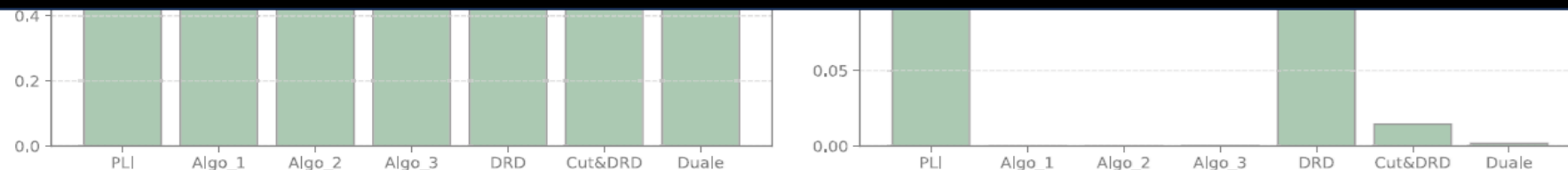


RISULTATI OTTENUTI – VERTEX COVER PESATO

- Algoritmo 2 (parte dal vertice meno denso): rispetto ad Algoritmo 1?

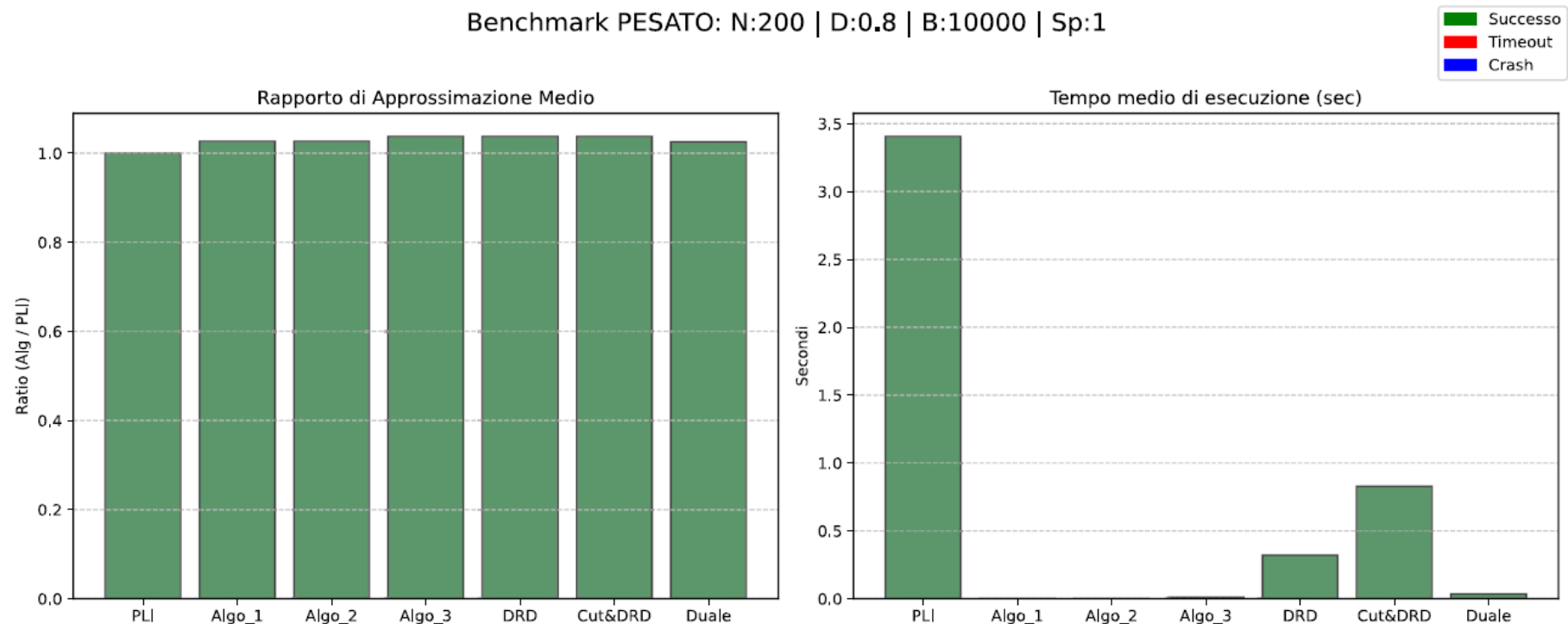


In questo caso $Sp = 1$, l'Algoritmo 1 è pesantemente sfavorito



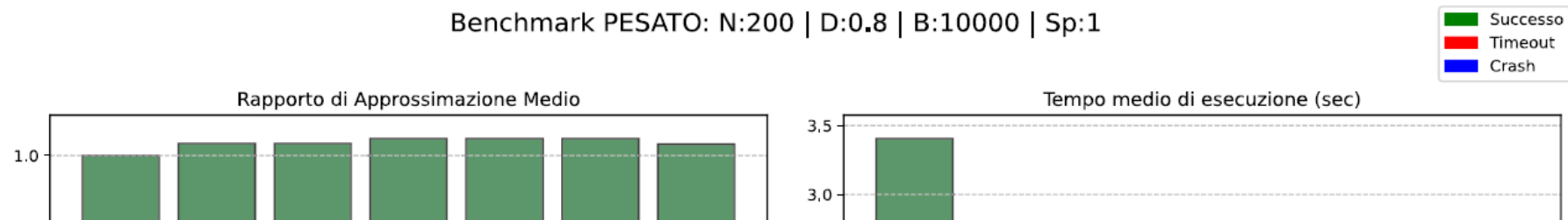
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Rapporti di approssimazione simili

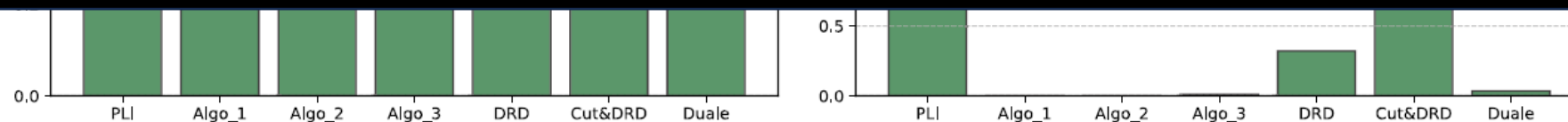


RISULTATI OTTENUTI – VERTEX COVER PESATO

- Rapporti di approssimazione simili

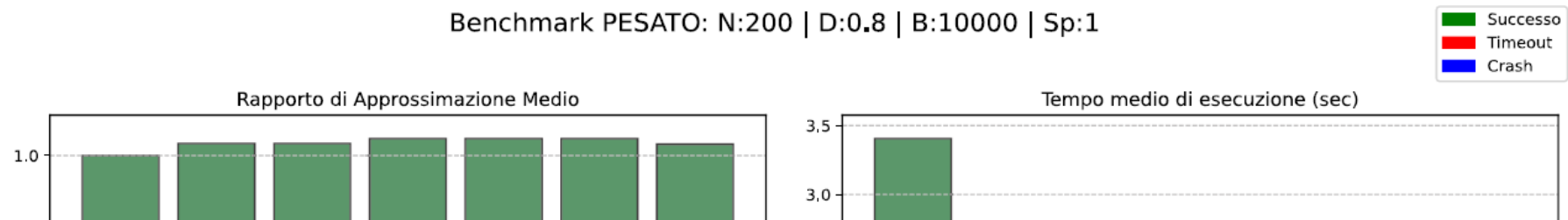


Se il grafo è molto denso o ha molti vertici, gli algoritmi offrono risultati mediamente simili

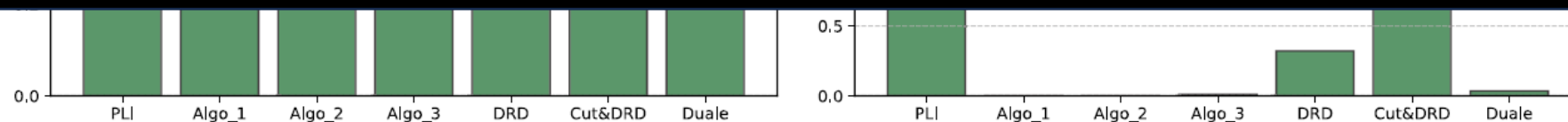


RISULTATI OTTENUTI – VERTEX COVER PESATO

- Rapporti di approssimazione simili

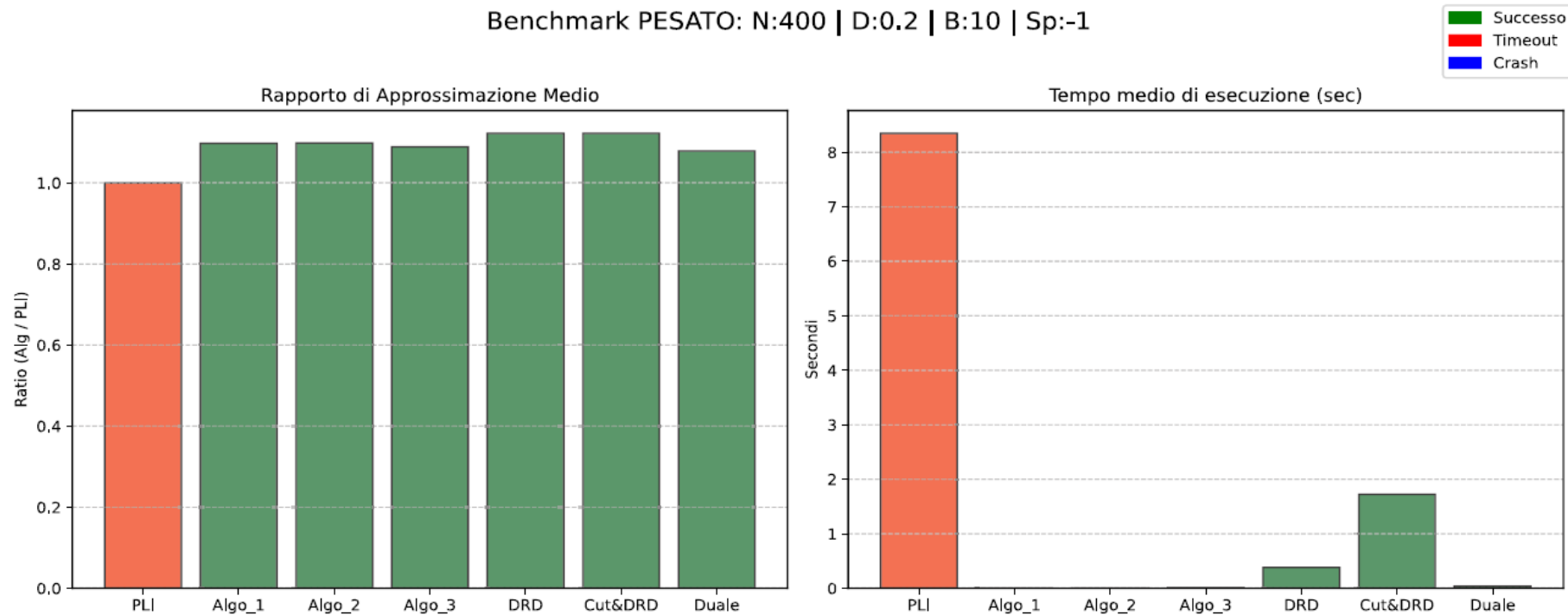


Meglio preferire algoritmi che non impiegano solver



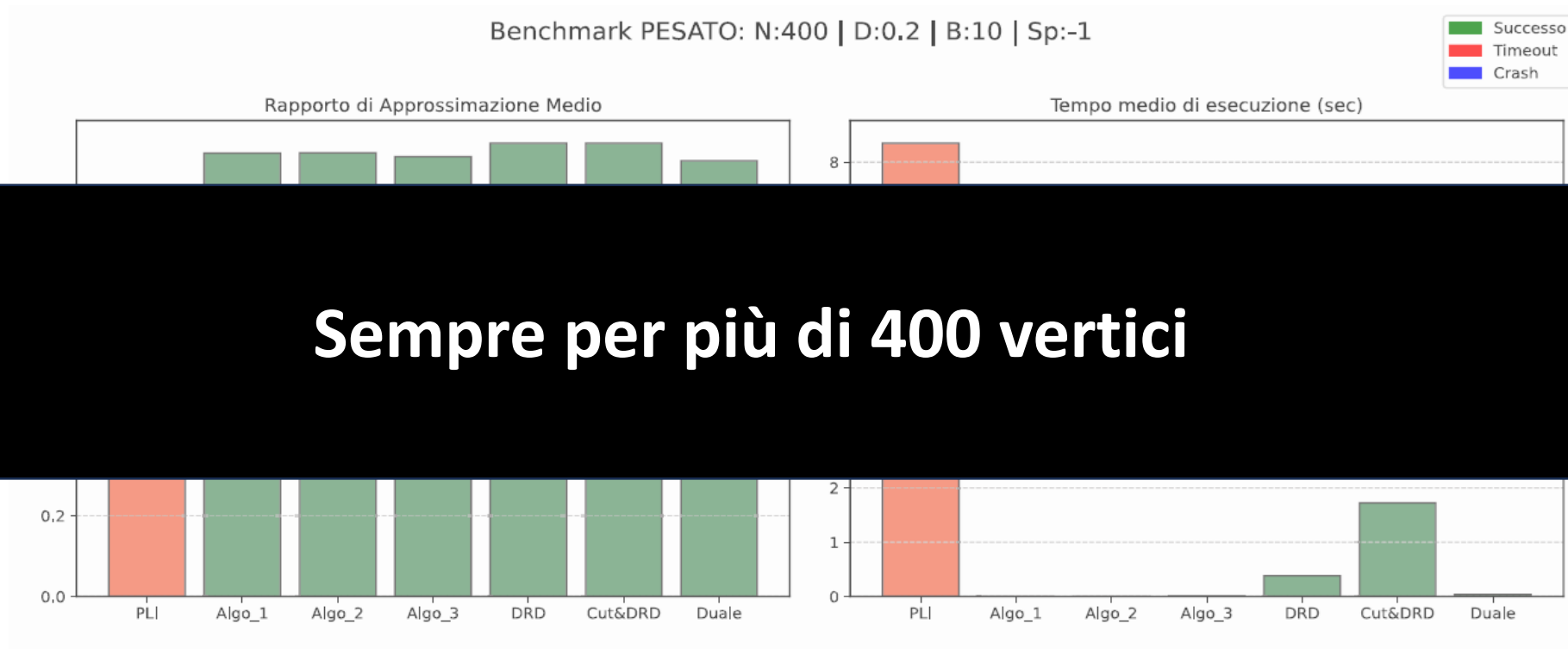
RISULTATI OTTENUTI – VERTEX COVER PESATO

- Timeout del Solver



RISULTATI OTTENUTI – VERTEX COVER PESATO

- Timeout del Solver



ROADMAP:

1. IDEA DEL PROGETTO

2. ALGORITMI CONFRONTATI

3. METODOLOGIA

4. RISULTATI PIÙ SIGNIFICATIVI

5. CONCLUSIONI

CONCLUSIONI

- Gli algoritmi esaminati hanno mostrato come l'utilizzo di **solver** sembri più raccomandato per **istanze piccole**, mentre l'impiego di algoritmi euristici potrebbe essere più indicato per istanze di grandi dimensioni. Per grafi con molti vertici e con grado «uniforme», le euristiche propongono risultati molto vicini all'ottimo e con tempi di esecuzione decisamente ridotti.

CONCLUSIONI

- Gli algoritmi basati su doubling and rounding down hanno inserito **in cover più vertici di quelli che avrebbero dovuto**, soffrendo l'elevata variabilità (di grado nel caso non pesato e di peso nel caso pesato) in modo evidente.
- Utilizzare le odd cycle inequalities apporta un vantaggio significativo solo quando i **triangoli sono facili da individuare** (pochi vertici con grado basso = meno possibilità da esplorare)

CONCLUSIONI

- **Materiale completo:**
 - Codice
 - Performance vertex cover non pesato
 - Performance vertex cover pesato
 - Questa presentazione

https://github.com/gianpaolo-pestilli/amod_project.git

